آموزش گام‌به‌گام زبان Rust

نگارش اول - مهر ۱۴۰۵

فهرست مطالب

بخش‌ها و ۶۹ درس

- مقدمه
- بخش اول: پایه‌های زبان
  - درس ۱: Hello World و ساختار پروژه
  - درس ۲: Comment ها
  - درس ۳: Identifier ها و Keyword ها
  - درس ۴: متغیرها
  - درس ۵: نوع‌های scalar و literal های عددی
  - درس ۶: عملگرها
  - درس ۷: تبدیل نوع با as
  - درس ۸: Tuple و unit
  - درس ۹: Array
  - درس ۱۰: Range و برش (slice)
  - درس ۱۱: Statement، Expression و Block
  - درس ۱۲: شرط‌ها با if
  - درس ۱۳: حلقه‌ها

- درس ۱۴: تابع‌ها
- درس ۱۵: const و static
- بخش دوم: مالکیت و قرض‌گیری
  - درس ۱۶: Ownership و Move
  - درس ۱۷: Copy، Clone و Drop
  - درس ۱۸: Reference و Borrowing
  - درس ۱۹: String، &str و literal های متنی
  - درس ۲۰: ماکروهای رایج و فرمت‌بندی
- بخش سوم: مدل‌سازی داده و ساختار برنامه
  - درس ۲۱: Struct
  - درس ۲۲: impl و متدها
  - درس ۲۳: Enum
  - درس ۲۴: match
  - درس ۲۵: Pattern ها (کامل)
  - درس ۲۶: if let، while let، let else و matches!
  - درس ۲۷: Option<T>
  - درس ۲۸: Result، panic! و عملگر ?
  - درس ۲۹: Module و visibility
  - درس ۳۰: Path ها و use
  - درس ۳۱: Crate و Cargo
  - درس ۳۲: Vec<T>
  - درس ۳۳: مجموعه‌ها (HashMap، HashSet، BTreeMap، BTreeSet، VecDeque)
  - درس ۳۴: ورودی/خروجی و محیط
- بخش چهارم: Generic، Trait و Lifetime
  - درس ۳۵: Generics
  - درس ۳۶: Trait

- درس ۳۷: Trait bound ها
- درس ۳۸: Attribute ها (مقدماتی) و derive
- درس ۳۹: Operator overloading
- درس ۴۰: Trait های تبدیل
- درس ۴۱: Associated type و associated const
- درس ۴۲: رفع ابهام در فراخوانی و supertrait
- درس ۴۳: impl Trait در خروجی و trait object
- درس ۴۴: Closure و function pointer
- درس ۴۵: Iterator
- درس ۴۶: Lifetime (پایه)
- درس ۴۷: Lifetime (پیشرفته)
- درس ۴۸: Smart pointer ها
- درس ۴۹: Interior mutability
- درس ۵۰: مدیریت خطای حرفه‌ای
- درس ۵۱: تست و مستندسازی
- درس ۵۲: Sized و DST
- بخش پنجم: همزمانی و Async
  - درس ۵۳: Thread ها
  - درس ۵۴: ارتباط و اشتراک داده بین thread ها
  - درس ۵۵: Async/await (پایه)
  - درس ۵۶: Async (پیشرفته)
- بخش ششم: Attribute و ماکرو
  - درس ۵۷: Attribute ها (عمیق) و کامپایل شرطی
  - درس ۵۸: نوشتن ماکرو با macro_rules!
  - درس ۵۹: Procedural macro
- بخش هفتم: سیستم نوع پیشرفته و سطح پایین

- درس ۶۰: Trait های پیشرفته
- درس ۶۱: نوع ! و تابع diverging
- درس ۶۲: Const evaluation
- درس ۶۳: الگوهای نوع
- درس ۶۴: Memory layout
- درس ۶۵: unsafe
- درس ۶۶: FFI (ارتباط با C)
- درس ۶۷: Inline assembly
- درس ۶۸: no_std و WebAssembly

- بخش هشتم: مرور نهایی و جدول‌های مرجع
- درس ۶۹: مرور نهایی نمادها و کلمات کلیدی Rust

به نام خدا

مقدمه

این کتاب Rust را در ۶۹ درس کوتاه، از صفر تا سطح پیشرفته آموزش می‌دهد. هر درس یک موضوع را با توضیح مختصر و مثال‌های کوچک و قابل‌اجرا بیان می‌کند، به‌طوری که بتوان آن را در یک نشست مطالعه کرد.

نشانی گیت‌هاب کتاب: https://github.com/amir-saniyan/rust-step-by-step
تهیه‌کننده: امیر سانیان

ترتیب درس‌ها و پوشش موضوعات

ترتیب درس‌ها عمداً از ساده به سخت است و به ترتیب سنتی کتاب‌های Rust وابسته نیست. هر درس فقط به مفاهیمی تکیه می‌کند که پیش‌تر آمده‌اند؛ اگر مفهومی از درس‌های بعدی لازم باشد، تنها در حد یک جمله معرفی می‌شود.

فهرست موضوعات بر پایه‌ی «Syntax index» در مرجع رسمی زبان (The Rust Reference) تنظیم شده است تا همه‌ی کلمه‌های کلیدی، عملگرها، توکن‌ها، ماکروها، attribute ها، expression ها، item ها، type ها و pattern ها در جایی از کتاب بیایند. مفاهیم مهمی که در آن فهرست نیستند، مانند ownership، lifetime، همزمانی و کتابخانه‌ی استاندارد، نیز به کتاب اضافه شده‌اند.

مخاطب و پیش‌نیاز

این کتاب برای برنامه‌نویسانی است که با مفاهیم عمومی برنامه‌نویسی (متغیر، شرط، حلقه، تابع، آرایه) در یک زبان دیگر آشنا هستند. آشنایی با C، C++ یا زبانی مشابه مفید است، اما الزامی نیست.

ساختار کتاب

1. بخش اول، پایه‌های زبان (درس ۱ تا ۱۵): ساختار پروژه، متغیرها، نوع‌های پایه، عملگرها، شرط و حلقه، تابع.
2. بخش دوم، مالکیت و قرض‌گیری (درس ۱۶ تا ۲۰): ownership، borrowing، رشته‌ها، ماکروهای رایج.
3. بخش سوم، مدل‌سازی داده و ساختار برنامه (درس ۲۱ تا ۳۴): pattern، enum، struct، مدیریت خطا، ماژول‌ها، مجموعه‌ها، ورودی/خروجی.
4. بخش چهارم، Trait، Generic و Lifetime (درس ۳۵ تا ۵۲).
5. بخش پنجم، همزمانی و Async (درس ۵۳ تا ۵۶).
6. بخش ششم، Attribute و ماکرو (درس ۵۷ تا ۵۹).
7. بخش هفتم، سیستم نوع پیشرفته و سطح پایین (درس ۶۰ تا ۶۸).
8. بخش هشتم، مرور نهایی و جدول‌های مرجع (درس ۶۹).

روش مطالعه

- درس‌ها را به ترتیب بخوانید؛ هر درس بر درس‌های قبلی بنا شده است.
- هر مثال را در یک پروژه‌ی `cargo` جداگانه اجرا کنید و کد را تغییر دهید تا رفتار آن را ببینید.
- پیام‌های کامپایلر Rust دقیق و آموزنده‌اند؛ آن‌ها را با دقت بخوانید.
- در پایان هر درس، بخش «اشتباه رایج» را بخوانید؛ این اشتباه‌ها بیشتر از همه برای تازه‌کارها پیش می‌آیند.

قراردادهای نگارشی

- نام‌های فنی، کلمه‌های کلیدی و نام نوع‌ها به انگلیسی نوشته می‌شوند.
- مثال‌های کد برای Rust نسخه‌ی ۱٫۸۵ یا جدیدتر و edition 2024 نوشته شده‌اند.
- مثال‌هایی که عمداً کامپایل نمی‌شوند، با کامنتی که با «خطای کامپایل» شروع می‌شود مشخص شده‌اند.
- بعضی مثال‌ها هشدار (warning) کامپایلر تولید می‌کنند، مثلاً برای متغیر استفاده‌نشده؛ این هشدارها خطا نیستند.

- بخش «اشتباه رایج» در پایان هر درس با علامت نقل‌قول مشخص شده است.

آماده‌سازی محیط

ابتدا Rust را با ابزار `rustup` از وب‌سایت رسمی rust-lang.org نصب کنید. سپس نصب را بررسی کنید:

```
rustc --version   # نسخه‌ی کامپایلر
cargo --version   # نسخه‌ی ابزار ساخت
rustup update     # به‌روزرسانی به آخرین نسخه‌ی پایدار
```

برای تمرین، یک پروژه‌ی آزمایشی بسازید و مثال‌ها را در فایل `src/main.rs` آن اجرا کنید:

```
cargo new playground
cd playground
cargo run
```

بخش اول: پایه‌های زبان

درس ۱: Hello World و ساختار پروژه

نصب و ساخت پروژه

برای شروع، Rust را با `rustup` نصب کنید. سپس در ترمینال:

```
cargo new hello_rust   # ساخت پروژه (در نسخه‌های جدید، edition 2024 پیش‌فرض است)
cd hello_rust
cargo run              # کامپایل و اجرا
```

- `cargo new` : ساخت پروژه با فایل‌های اولیه.
- `cargo run` : کامپایل و اجرای برنامه.
- `cargo build` : کامپایل بدون اجرا.
- `cargo check` : بررسی خطاهای کامپایل بدون تولید فایل اجرایی؛ سریع‌تر از `build` .

تابع main و چاپ متن

فایل `src/main.rs` نقطه‌ی شروع برنامه است:

```
fn main() {
    // چاپ پیام در ترمینال
    println!("Hello, world!");
}
```

- `fn` برای تعریف تابع است.
- `main` تابعی است که اجرای برنامه از آن آغاز می‌شود.
- `println!` متن را چاپ می‌کند و به خط بعد می‌رود. علامت `!` نشان می‌دهد که `println!` یک ماکرو است، نه تابع معمولی؛ ماکروها در درس ۲۰ بررسی می‌شوند.

قراردادهای نام‌گذاری

- `snake_case`: حروف کوچک با `_`؛ برای توابع و متغیرها.
- `UpperCamelCase`: حرف اول هر کلمه بزرگ؛ برای نوع‌ها، مانند `UserProfile`.

```rust
fn print_message() {
    // snake_case نام تابع با
    println!("Hello, Rust!");
}

struct UserProfile {
    age: u32, // UpperCamelCase نام نوع با
}

fn main() {
    print_message();
    let user = UserProfile { age: 25 };
    println!("{}", user.age);
}
```

اشتباه رایج: استفاده از `cargo check` به‌جای `cargo run` برای اجرای برنامه؛ `check` فقط خطاها را بررسی می‌کند و برنامه را اجرا نمی‌کند.

درس ۲: Comment ها

کامنت‌های معمولی

کامنت‌ها برای توضیح کد هستند و در اجرای برنامه اثری ندارند.

```rust
fn main() {
    // کامنت تکخطی
    println!("Hello!");

    /* کامنت چندخطی
       که می‌تواند چند خط داشته باشد */

    /* کامنت تو در تو:
       /* این هم یک کامنت داخلی است */ */
    */
}
```

- `//` : کامنت تک‌خطی.
- `/* ... */` : کامنت بلوکی؛ می‌تواند تو در تو باشد.

مستندسازی با doc comment

Doc comment ها به مستندات برنامه تبدیل می‌شوند. دستور `cargo doc --open` مستندات HTML را تولید می‌کند و باز می‌کند.

```rust
/// این تابع دو عدد را جمع می‌کند.
fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    println!("{}", add(2, 3));
}
```

- `///` و `/** ... */`: مستندات **outer**؛ آیتمی را که بعد از کامنت می‌آید، مثل یک تابع، توضیح می‌دهند.
- `//!` و `/*! ... */`: مستندات **inner**؛ آیتمی را که کامنت درون آن قرار دارد، مثل کل crate یا یک ماژول، توضیح می‌دهند.

مثال مستندات inner در ابتدای `src/main.rs`:

```rust
//! این فایل نقطه‌ی شروع برنامه است.
//! این توضیح، مستندات خود crate محسوب می‌شود.

fn main() {
    println!("Hello!");
}
```

> **اشتباه رایج:** استفاده از `//` به‌جای `///` برای مستندسازی؛ کامنت معمولی وارد مستندات API نمی‌شود.

درس ۳: Identifier ها و Keyword ها

قواعد identifier

Identifier نام متغیر، تابع، نوع و موارد مشابه است.

- با حرف یا `_` شروع می‌شود؛ بعد از آن حروف، رقم و `_` مجازند.
- به بزرگی و کوچکی حروف حساس است.
- برای توابع و متغیرها `snake_case` و برای نوع‌ها `UpperCamelCase` رایج است.
- Keyword ها را نمی‌توان مستقیماً به‌عنوان نام به کار برد؛ پیشوند `r#` (raw identifier) این محدودیت را برطرف می‌کند.

```
fn main() {
    let r#match = 10; // match یک keyword است؛ با r# می‌توان از آن به‌عنوان نام استفاده کرد
    let r#gen = 42;   // gen در edition 2024 رزرو شده است
    let user_age = 25;
    println!("{} {} {}", r#match, r#gen, user_age);
}
```

کلمه‌های `self`، `Self`، `super` و `crate` را نمی‌توان با `r#` به‌عنوان identifier به کار برد.

Keyword های فعال

- کنترل جریان: `if`، `else`، `match`، `loop`، `while`، `for`، `in`، `break`، `continue`، `return`
- تعریف آیتم‌ها: `fn`، `struct`، `enum`، `trait`، `impl`، `type`، `mod`، `use`، `const`، `static`
- متغیر و مقدار: `let`، `mut`، `ref`، `move`، `true`، `false`
- مسیرها: `self`، `Self`، `super`، `crate`

- سایر: `pub` ، `as` ، `dyn` ، `unsafe` ، `extern` ، `where` ، `async` ، `await`

Keyword های رزروشده، contextual و توکن‌های بلااستفاده

کلمه‌های `abstract` ، `become` ، `box` ، `do` ، `final` ، `gen` ، `macro` ، `override` ، `priv` ، `try` ، `typeof` ، `unsized` ، `virtual` ، `yield` برای کاربردهای احتمالی آینده رزرو شده‌اند و فعلاً نمی‌توان از آن‌ها به‌عنوان نام استفاده کرد، مگر با `r#` .

چند کلمه فقط در زمینه‌ی خاصی معنای ویژه دارند (contextual) و keyword عمومی نیستند: `macro_rules` (تعریف ماکرو)، `union` ، `raw` و `safe` . همچنین `'static` یک نام lifetime است، نه keyword.

توکن‌های `<-` و `~` از قبل از Rust 1.0 بلااستفاده مانده‌اند، اما همچنان توکن شناخته می‌شوند.

> **اشتباه رایج:** تصور اینکه هر واژه‌ی خاصی مثل `macro_rules` یا `union` یک keyword عمومی است؛ این کلمه‌ها فقط در زمینه‌ی خاص خود معنا دارند. در عوض `is` در Rust اصلاً keyword نیست.

درس ۴: متغیرها

تعریف متغیر و تغییرپذیری

متغیرها با `let` تعریف می‌شوند و به‌طور پیش‌فرض immutable (تغییرناپذیر) هستند. برای اینکه مقدار قابل تغییر باشد، از `mut` استفاده کنید.

```
fn main() {
    let x = 10;       // تغییرناپذیر
    let mut y = 20;   // تغییرپذیر

    y = 30;
    println!("{x}, {y}");
}
```

تعیین نوع و استنتاج نوع

کامپایلر معمولاً نوع متغیر را از مقدار آن تشخیص می‌دهد (type inference). در صورت نیاز می‌توان نوع را صریح نوشت (type annotation).

```
fn main() {
    let age: u32 = 25; // تعیین صریح نوع
    let price = 12.5;  // استنتاج می‌شود f64 نوع

    println!("{age}, {price}");
}
```

Shadowing و متغیرهای نادیده‌گرفته‌شده

با تعریف دوباره‌ی یک نام با `let`، متغیر جدیدی ساخته می‌شود که متغیر قبلی را پنهان می‌کند؛ به این کار shadowing می‌گویند. در این حالت حتی نوع متغیر می‌تواند عوض شود.

```
fn main() {
    let value = "42";
    let value = value.len(); // تغییر می‌کند usize به str& نوع از ؛shadowing

    let _ = 100;       // مقدار عمداً نادیده گرفته می‌شود
    let _unused = 200; // متغیری معمولی که کامپایلر درباره‌ی استفاده‌نشدنش هشدار نمی‌دهد

    println!("{value}, {_unused}");
}
```

- `mut`: همان متغیر تغییر می‌کند و نوعش ثابت می‌ماند.
- shadowing: متغیر جدیدی با همان نام تعریف می‌شود.
- `_`: مقدار را کاملاً نادیده می‌گیرد و هیچ متغیری نمی‌سازد.
- `_name`: یک متغیر معمولی است و قابل استفاده است؛ فقط هشدار «استفاده‌نشده» را خاموش می‌کند.

> **اشتباه رایج:** تصور اینکه `_name` مثل `_` کاملاً نادیده گرفته می‌شود؛ `_name` همچنان یک متغیر است.

درس ۵: نوع‌های scalar و literal های عددی

نوع‌های scalar

نوع‌های scalar یک مقدار منفرد نگه می‌دارند:

- صحیح علامت‌دار: `i8` , `i16` , `i32` , `i64` , `i128` ؛ اعداد مثبت و منفی.
- صحیح بدون علامت: `u8` , `u16` , `u32` , `u64` , `u128` ؛ اعداد نامنفی.
- وابسته به معماری: `isize` , `usize` ؛ اندازه‌شان با معماری هدف (معمولاً ۳۲ یا ۶۴ بیت) برابر است.
- اعشاری: `f32` , `f64` ؛ `f64` پیش‌فرض است.
- منطقی: `bool` با مقدارهای `true` و `false` .
- کاراکتر: `char` ؛ یک مقدار Unicode، مانند `'ع'` .

Literal های عددی

```
fn main() {
    let a = 10u8;       // پسوند نوع
    let b = 2.5f32;     // f32 عدد اعشاری
    let c = 1_000;      // جداکننده برای خوانایی
    let hex = 0xff;     // مبنای ۱۶
    let oct = 0o77;     // مبنای ۸
    let bin = 0b1010;   // مبنای ۲
    let ch = b'a';      // ASCII با مقدار byte literal

    println!("{a} {b} {c} {hex} {oct} {bin} {ch}");
}
```

`b'a'` از نوع `u8` است، نه `char` ؛ برای یک کاراکتر Unicode از `'a'` استفاده کنید.

Overflow

```
fn main() {
    let x: u8 = 255;

    // در حالت `debug`، `x + 1` باعث panic می‌شود (overflow).
    // در حالت `release`، به‌طور پیش‌فرض wrap می‌شود (برابر 0).

    // برای رفتار قطعی از متدهای صریح استفاده کنید:
    println!("{}", x.wrapping_add(1));   // 0
    println!("{:?}", x.checked_add(1));  // None
    println!("{}", x.saturating_add(1)); // 255
}
```

> **اشتباه رایج:** فرض اینکه `u8` می‌تواند عدد ۲۵۶ را نگه دارد؛ بازه‌ی آن `0..=255` است.

درس ۶: عملگرها

عملگرهای حسابی و مقایسه‌ای

```
fn main() {
    let a = 10;
    let b = 3;

    // عملگرهای حسابی
    println!("{} {} {} {} {}", a + b, a - b, a * b, a / b, a % b);

    // مقایسه؛ نتیجه bool است
    println!("{} {}", a > b, a == b);
    println!("{}", -a); // منفی‌کردن عدد
}
```

تقسیم دو عدد صحیح، بخش اعشاری را حذف می‌کند. عملگرهای مقایسه `==`، `!=`، `<`، `>`، `<=`، `>=` هستند.

عملگرهای منطقی و بیتی

```rust
fn main() {
    let a = true;
    let b = false;

    // ارزیابی تنبل: اگر طرف چپ `&&` نادرست باشد، طرف راست بررسی نمی‌شود
    println!("{} {} {}", a && b, a || b, !a);

    let x: u8 = 0b1100;
    let y: u8 = 0b1010;
    println!("{:04b} {:04b} {:04b}", x & y, x | y, x ^ y);
    println!("{:08b} {:08b}", !x, x << 1);
}
```

عملگرهای بیتی `&`، `|`، `^`، `!`، `<<`، `>>` هستند. `&&` و `||` ارزیابی تنبل دارند؛ `&` و `|` روی اعداد، عملیات بیتی انجام می‌دهند.

انتساب ترکیبی و اولویت

```rust
fn main() {
    let mut x = 10;
    x += 5;  // 15
    x *= 2;  // 30
    x ^= 3;  // 29 :بیتی XOR
    println!("{x}");

    println!("{}", 2 + 3 * 4);   // 14
    println!("{}", (2 + 3) * 4); // 20
}
```

اولویت عملگرها از بالا به پایین:

1. عملگرهای یکانی: `-`, `!`, `*` (dereference), `&`
2. `as`
3. `*`, `/`, `%`
4. `+`, `-`
5. `<<`, `>>`
6. `&`
7. `^`
8. `|`
9. مقایسه‌ها: `==`, `!=`, `<`, `>`, `<=`, `>=`
10. `&&`
11. `||`
12. `..` و `..=`

13. انتساب: `=` و انتساب‌های ترکیبی

خلاصه‌ی عملگرها

- حسابی: `+ - * / %`
- مقایسه‌ای: `== != < > <= >=`
- منطقی: `&&` و `||` (ارزیابی تنبل) و `!`
- بیتی: `&` ، `|` ، `^` ، `!` ، `<<` ، `>>`
- انتساب ترکیبی: `+= -= *= /= %= ^= &= |= <<= >>=`
- منفی‌سازی: `-expr`

> **اشتباه رایج:** اشتباه گرفتن `&&` منطقی با `&` بیتی؛ نوع داده و هدف عملیات متفاوت است.

درس ۷: تبدیل نوع با as

ساختار تبدیل

عبارت `expr as Type` مقدار را به نوع دیگری تبدیل می‌کند.

```
fn main() {
    let x: i32 = 300;
    let y = x as u8;
    // بیت‌های اضافی حذف می‌شوند (truncation): 300 به 44 تبدیل می‌شود
    println!("{y}");

    let a: i8 = -1;
    let b = a as u16;
    // مقدار ابتدا با حفظ علامت گسترش می‌یابد (sign extension) و سپس بدون علامت خوانده می‌شود: 65535
    println!("{b}");
}
```

- Truncation: هنگام تبدیل به نوع صحیح کوچک‌تر، بیت‌های اضافی حذف می‌شوند.
- Sign extension: هنگام گسترش عدد صحیح علامت‌دار، بیت علامت حفظ می‌شود؛ بنابراین `-1i8` پس از تبدیل به `u16` برابر `65535` است.

تبدیل اعشاری و نوع‌های دیگر

```
fn main() {
    let x = 12.9f32 as i32;   // بخش اعشاری حذف می‌شود: 12
    let y = -12.9f32 as u32;  // مقدار منفی به 0 محدود می‌شود
    let c = 'A' as u32;       // Unicode: 65 مقدار
    let b = true as u8;       // true برابر 1 و false برابر 0

    println!("{x} {y} {c} {b}");
}
```

تبدیل `float` به `int` با `as` مقدار را به سمت صفر گرد می‌کند و نتیجه را در بازه‌ی نوع مقصد نگه می‌دارد (saturating).

تبدیل ایمن‌تر

برای تبدیل‌هایی که نباید بی‌صدا مقدار را تغییر دهند، از این‌ها استفاده کنید (در درس ۴۰ کامل‌تر بررسی می‌شوند):

- `From` و `Into`: تبدیل‌های مستقیم که هرگز شکست نمی‌خورند.
- `TryFrom` و `TryInto`: تبدیل‌هایی که ممکن است شکست بخورند.

```
fn main() {
    let big: i32 = 300;
    println!("{:?}", u8::try_from(big)); // چون 300 در u8 جا نمی‌شود Err(...)
    println!("{:?}", u8::try_from(200)); // Ok(200)
}
```

> **اشتباه رایج:** فرض اینکه `as` هنگام خارج‌بودن مقدار از بازه‌ی نوع مقصد خطا می‌دهد؛ `as` بی‌صدا مقدار را کوتاه یا محدود می‌کند.

درس ۸: Tuple و unit

ساخت tuple و دسترسی به اعضا

Tuple مجموعه‌ای با تعداد ثابت از مقدارهاست که می‌توانند نوع‌های متفاوت داشته باشند.

```
fn main() {
    let person = ("Ali", 25, true);

    // دسترسی با اندیس؛ اندیس از صفر شروع می‌شود
    println!("{} {}", person.0, person.1);
    println!("{}", person.2);
}
```

Tuple تک‌عضوی و destructuring

برای ساخت tuple تک‌عضوی، ویرگول پایانی لازم است. بدون آن، پرانتز فقط یک عبارت را گروه‌بندی می‌کند.

```
fn main() {
    let one = (42,);    // tuple تک‌عضوی
    let number = (42);  // فقط عدد؛ کامپایلر درباره‌ی پرانتز اضافی هشدار می‌دهد

    // (destructuring) بازکردن tuple و قراردادن اعضا در متغیرها
    let (name, age) = ("Sara", 30);

    println!("{} {} {}", one.0, number, name);
    println!("{age}");
}
```

نوع unit

نوع `()` که به آن unit می‌گویند، هیچ داده‌ای نگه نمی‌دارد و برای نشان‌دادن «نبودن مقدار» به کار می‌رود. تابعی که مقدار دیگری برنمی‌گرداند، به‌طور پیش‌فرض `()` برمی‌گرداند.

```
fn greet() {
    // این تابع مقدار مشخصی برنمی‌گرداند
    println!("Hello!");
}

fn main() {
    let result: () = greet();
    println!("{:?}", result); // ()
}
```

> **اشتباه رایج:** نوشتن `(42)` به‌جای `(42,)`؛ اولی یک عدد است، نه tuple.

درس ۹: Array

تعریف و ساخت array

Array مجموعه‌ای از عناصر هم‌نوع با طول ثابت است. نوع آن به شکل `[Type; len]` نوشته می‌شود.

```rust
fn main() {
    // ساخت آرایه با عناصر مشخص
    let numbers: [i32; 3] = [10, 20, 30];

    // دسترسی با اندیس؛ اندیس از صفر شروع می‌شود
    println!("{} {}", numbers[0], numbers[2]);

    // تعداد عناصر آرایه
    println!("{}", numbers.len());
}
```

مقداردهی تکراری و پیمایش

```rust
fn main() {
    // آرایه‌ای با پنج عنصر مساوی
    let zeros = [0; 5];

    // پیمایش ساده با for
    for value in zeros {
        println!("{value}");
    }
}
```

در `[0; 5]`، مقدار `0` پنج بار تکرار می‌شود. طول آرایه در زمان کامپایل مشخص است و در زمان اجرا تغییر نمی‌کند.

دسترسی خارج از محدوده

```
fn main() {
    let data = [10, 20, 30];

    // اندیس را در زمان اجرا می‌سازیم؛ اگر اندیس ثابتِ نامعتبر باشد،
    // کامپایلر همان لحظه خطا می‌دهد.
    let index = std::env::args().count() + 2; // معمولاً 3
    println!("{}", data[index]); // panic: index out of bounds

    // روش ایمن: get مقدار Option برمی‌گرداند
    println!("{:?}", data.get(index)); // None
}
```

اندیس‌های معتبر این آرایه `0` تا `2` هستند. دسترسی خارج از محدوده در زمان اجرا `panic` می‌کند.

> **اشتباه رایج:** تصور اینکه آرایه می‌تواند عناصر با نوع‌های متفاوت یا طول متغیر داشته باشد؛ همه‌ی عناصر باید هم‌نوع باشند و طول ثابت است (برای طول متغیر از `Vec` استفاده می‌شود؛ درس ۳۲).

درس ۱۰: Range و برش (slice)

ساخت range

Range بازه‌ای از مقدارها را مشخص می‌کند. `..` انتهای بازه را شامل نمی‌شود و `..=` آن را شامل می‌شود.

```
fn main() {
    // از 2 تا قبل از 5
    for n in 2..5 {
        println!("{n}"); // 4 ،3 ،2
    }

    // از 2 تا خود 5
    for n in 2..=5 {
        println!("{n}"); // 5 ،4 ،3 ،2
    }
}
```

شکل‌های مختلف range:

- `a..b`: از `a` تا قبل از `b`
- `a..=b`: از `a` تا خود `b`
- `a..`: از `a` به بعد
- `..b`: از ابتدا تا قبل از `b`
- `..=b`: از ابتدا تا خود `b`
- `..`: کل بازه

برش (slice) آرایه

Slice بخشی از آرایه را بدون کپی‌کردن عناصر نشان می‌دهد. نوع آن معمولاً `&[T]` است که `T` نوع عناصر است. علامت `&` را در درس ۱۸ کامل می‌آموزید؛ فعلاً آن را «نگاهی به بخشی از داده» بدانید.

```
fn main() {
    let numbers = [10, 20, 30, 40, 50];

    // شامل اندیس‌های 1، 2 و 3
    let part: &[i32] = &numbers[1..4];
    println!("{part:?}");

    // از ابتدا تا قبل از اندیس 2
    println!("{:?}", &numbers[..2]);

    // کل آرایه به‌شکل slice
    println!("{:?}", &numbers[..]);
}
```

for با Range

```
fn main() {
    // از صفر تا سه، بدون خود سه
    for i in 0..3 {
        println!("{i}");
    }
}
```

اشتباه رایج: در `numbers[a..b]`، اندیس `b` جزو slice نیست؛ همچنین اندیس‌های خارج از محدوده باعث `panic` می‌شوند.

درس ۱۱: Statement، Expression و Block

تفاوت‌ها

- Expression مقداری تولید می‌کند؛ مانند `2 + 3` یا `if true { 1 } else { 2 }`.
- Statement یک دستور است؛ مانند `let x = 5;` یا یک expression که با `;` تمام می‌شود.
- Block مجموعه‌ای از دستورها و عبارت‌ها درون `{ ... }` است و خودش هم یک expression است.
- آخرین expression بلوک، اگر `;` نداشته باشد، مقدار بلوک است.
- افزودن `;` به یک expression مقدار آن را کنار می‌گذارد و نتیجه `()` می‌شود.
- دستور `let` یک statement است و مقدار قابل‌استفاده تولید نمی‌کند.

مثال ۱: مقدار بلوک

```
fn main() {
    let x = {
        let a = 10; // دستور
        a + 5       // ; بدون؛ مقدار بلوک
    };

    println!("{x}"); // 15
}
```

مثال ۲: تأثیر ;

```
fn main() {
    let a = { 3 + 4 };   // مقدار از نوع i32 و برابر 7
    let b = { 3 + 4; };  // مقدار () یا unit

    println!("{a}, {b:?}");
}
```

مثال ۳: مقدار پیش‌فرض ()

```
fn main() {
    let result = {
        println!("Hello"); // این دستور مقدار بلوک را تعیین نمی‌کند
    };

    // چون بلوک expression نهایی ندارد، مقدار آن () است
    let empty: () = result;
    println!("{empty:?}");
}
```

> **اشتباه رایج:** گذاشتن `;` بعد از آخرین expression بلوک، مقدار آن را به `()` تبدیل می‌کند.

درس ۱۲: شرط‌ها با if

if / else if / else

شرط در Rust حتماً باید از نوع `bool` باشد؛ برخلاف برخی زبان‌ها، عددی مثل `1` به‌عنوان شرط پذیرفته نمی‌شود.

```
fn main() {
    let n = 7;

    if n < 0 {
        println!("منفی");
    } else if n == 0 {
        println!("صفر");
    } else {
        println!("مثبت");
    }
}
```

if به‌عنوان expression

چون `if` می‌تواند مقدار تولید کند، می‌توان نتیجه‌اش را در یک متغیر ذخیره کرد. همه‌ی شاخه‌ها باید نوع یکسانی داشته باشند.

```
fn main() {
    let age = 20;

    // هر دو شاخه مقدار i32 تولید می‌کنند
    let status = if age >= 18 { 1 } else { 0 };

    println!("{status}");
}
```

شاخه‌های با نوع ناسازگار

این مثال عمداً کامپایل نمی‌شود:

```
fn main() {
    let n = 5;

    // خطای کامپایل: یک شاخه i32 و شاخه‌ی دیگر &str برمی‌گرداند
    // ;{ "منفی یا صفر" } let result = if n > 0 { 10 } else
}
```

نوع نتیجه‌ی همه‌ی شاخه‌ها باید یکسان باشد.

> **اشتباه رایج:** فراموش‌کردن `else` هنگام استفاده از `if` برای تولید مقداری که باید در همه‌ی حالت‌ها وجود داشته باشد.

درس ۱۳: حلقه‌ها

loop و while

`loop` تا اجرای `break` ادامه می‌یابد. `while` تا وقتی شرط `bool` آن درست است اجرا می‌شود.

```
fn main() {
    let mut n = 0;

    // حلقه تا رسیدن به break ادامه دارد
    loop {
        n += 1;
        if n == 3 {
            break;
        }
    }

    // تا زمانی که شرط درست است
    while n > 0 {
        println!("{n}"); // 1 ،2 ،3
        n -= 1;
    }
}
```

for، continue و break با مقدار

`for pattern in expr` عناصر یک مجموعه یا بازه را پیمایش می‌کند. `continue` به تکرار بعدی می‌رود و `break` حلقه را متوقف می‌کند. `break` در `loop` می‌تواند مقدار برگرداند.

```
fn main() {
    // پیمایش بازه از صفر تا قبل از پنج
    for n in 0..5 {
        if n == 2 {
            continue; // عدد 2 چاپ نمی‌شود
        }
        if n == 4 {
            break; // حلقه متوقف می‌شود
        }
        println!("{n}"); // 3 ،1 ،0
    }

    // دریافت مقدار از loop
    let result = loop {
        break 42;
    };
    println!("{result}");
}
```

برچسب حلقه (loop label)

با برچسبی مثل `'outer` می‌توان `break` یا `continue` را به حلقه‌ی بیرونی نسبت داد.

```
fn main() {
    'outer: for i in 1..4 {
        for j in 1..4 {
            if i == 2 && j == 2 {
                break 'outer; // خروج از حلقه‌ی بیرونی
            }
            println!("{i}, {j}");
        }
    }
}
```

اشتباه رایج: نوشتن `break;` بدون مقدار در `loop` ای که قرار است نتیجه بدهد؛ چون نوع نتیجه `()` می‌شود، با نوع مورد انتظار ناسازگار است و کامپایلر خطا می‌دهد.

درس ۱۴: تابع‌ها

تعریف تابع و پارامترها

با `fn` تابع تعریف می‌شود. نوع هر پارامتر باید مشخص باشد و `->` نوع مقدار برگشتی را تعیین می‌کند.

```
// دو عدد می‌گیرد و حاصل جمع را برمی‌گرداند
fn add(a: i32, b: i32) -> i32 {
    a + b // آخرین expression مقدار برگشتی است
}

fn main() {
    println!("{}", add(3, 4)); // () فراخوانی تابع با
}
```

return و بازگشت زودهنگام

`return` مقدار را برمی‌گرداند و اجرای تابع را همان‌جا تمام می‌کند. برای بازگشت زودهنگام (early return) می‌توان از آن استفاده کرد.

```
fn divide(a: i32, b: i32) -> i32 {
    if b == 0 {
        return 0; // بازگشت زودهنگام
    }

    a / b
}

fn main() {
    println!("{}", divide(10, 2)); // 5
    println!("{}", divide(10, 0)); // 0
}
```

تابع تودرتو

می‌توان داخل یک تابع، تابع دیگری تعریف کرد. تابع داخلی فقط در همان محدوده قابل دسترسی است.

```
fn main() {
    fn square(n: i32) -> i32 {
        n * n
    }

    // فراخوانی تابع داخلی
    println!("{}", square(5)); // 25
}
```

اشتباه رایج: گذاشتن `;` بعد از آخرین expression تابعی که باید مقداری غیر از `()` برگرداند؛ در این حالت مقدار دور ریخته می‌شود و کامپایلر خطای ناسازگاری نوع می‌دهد.

درس ۱۵: const و static

تعریف const

`const` برای تعریف مقدار ثابت در زمان کامپایل است. نوع آن باید نوشته شود و نام‌گذاری معمولاً با `SCREAMING_SNAKE_CASE` انجام می‌شود.

```rust
const MAX_SIZE: usize = 100;
const PI: f64 = 3.14159;

fn main() {
    // استفاده از مقدارهای ثابت
    let area = PI * 2.0 * 2.0;
    println!("{MAX_SIZE}, {area}");
}
```

تفاوت static و const

`static` یک جایگاه مشخص در حافظه دارد و در تمام اجرای برنامه زنده است. `const` مثل یک مقدار ثابت در هر محل استفاده درج می‌شود و لزوماً آدرس یکتا ندارد.

```rust
static APP_NAME: &str = "RustApp";
const VERSION: u32 = 1;

fn main() {
    println!("{APP_NAME} v{VERSION}");

    // هر دو ارجاع به همان جایگاه حافظه اشاره می‌کنند
    println!("{}", std::ptr::eq(&APP_NAME, &APP_NAME)); // true
}
```

بررسی شرط در زمان کامپایل

با `const _: () = ...` می‌توان شرطی را در زمان کامپایل بررسی کرد. اگر شرط برقرار نباشد، کامپایل شکست می‌خورد.

```
const BUFFER_SIZE: usize = 256;

// اگر شرط برقرار نباشد، برنامه کامپایل نمی‌شود
const _: () = assert!(BUFFER_SIZE.is_power_of_two());

fn main() {
    println!("{BUFFER_SIZE}");
}
```

`static mut` برای داده‌ی سراسری قابل‌تغییر است، اما دسترسی به آن ناامن است (در درس ۶۵ بررسی می‌شود). در کد معمولی از آن پرهیز کنید.

> **اشتباه رایج:** یکسان‌دانستن `const` و `static`؛ `static` جایگاه حافظه‌ی مشخصی دارد، اما `const` الزاماً آدرس یکتا ندارد.

بخش دوم: مالکیت و قرض‌گیری

مالکیت (ownership) ویژگی‌ای است که Rust را از بیشتر زبان‌ها متمایز می‌کند: مدیریت حافظه بدون garbage collector و بدون مدیریت دستی. مفاهیم این بخش پایه‌ی همه‌ی درس‌های بعدی‌اند؛ اگر چیزی در آن‌ها مبهم ماند، قبل از ادامه دوباره مرورشان کنید.

درس ۱۶: Ownership و Move

حافظه و قوانین ownership

- Stack: معمولاً برای مقدارهای ساده با اندازه‌ی مشخص و ثابت.
- Heap: برای داده‌هایی با اندازه یا طول متغیر؛ مانند `String` و `Vec`.

سه قانون ownership:

1. هر مقدار یک مالک (owner) دارد.
2. در هر لحظه فقط یک مالک وجود دارد.
3. وقتی مالک از محدوده‌اش (scope) خارج شود، مقدار آزاد می‌شود.

انتقال مالکیت (move)

در انتساب یک `String` یا `Vec` به متغیر دیگر، مالکیت منتقل می‌شود و متغیر قبلی دیگر قابل استفاده نیست.

```
fn main() {
    let first = String::from("Rust");
    let second = first; // مالکیت به second منتقل می‌شود

    println!("{second}");

    // (use after move) خطای کامپایل: first دیگر مالک نیست
    // println!("{first}");
}
```

انتقال به تابع و آزادسازی خودکار

ارسال مقدار به تابع نیز مالکیت را منتقل می‌کند. با پایان محدوده، مقدارِ متعلق به آن آزاد می‌شود (`Drop`).

```
fn show(items: Vec<i32>) {
    println!("{items:?}");
    // با پایان تابع، items آزاد می‌شود
}

fn main() {
    let numbers = vec![10, 20, 30];

    show(numbers); // مالکیت به تابع منتقل می‌شود

    // خطای کامپایل: numbers دیگر قابل استفاده نیست
    // println!("{numbers:?}");
}
```

اشتباه رایج: استفاده از `String` یا `Vec` پس از انتقال مالکیت؛ این کار خطای کامپایل `use of moved value` ایجاد می‌کند.

درس ۱۷: Copy، Clone و Drop

در این درس trait را فقط به‌معنای «قابلیتی که یک نوع دارد» به کار می‌بریم؛ trait ها در درس ۳۶ کامل بررسی می‌شوند.

نوع‌های Copy

`Copy` قابلیتی است که اجازه می‌دهد مقدار هنگام انتساب یا ارسال به تابع به‌طور ضمنی کپی شود؛ در نتیجه مالکیت منتقل نمی‌شود. اعداد، `bool`، `char` و tuple هایی که همه‌ی اعضایشان `Copy` باشند، `Copy` هستند.

```
fn main() {
    let a = 10;
    let b = a; // کپی ضمنی

    println!("{a}, {b}"); // هر دو معتبرند

    let point = (3, 4);
    let other = point; // است Copy ،از اعداد tuple
    println!("{point:?}, {other:?}");
}
```

تفاوت Copy و Clone

متد `clone()` کپی را به‌طور صریح انجام می‌دهد. `String` از نوع `Copy` نیست، چون داده‌اش روی heap است و کپی‌کردن آن به تخصیص حافظه و کپی محتوا نیاز دارد.

```
fn main() {
    let first = String::from("Rust");
    let second = first.clone(); // کپی صریح محتوا

    println!("{first}, {second}"); // هر دو معتبرند
}
```

ترتیب Drop

وقتی متغیرها از محدوده خارج می‌شوند، `drop` آن‌ها به ترتیب **معکوس** تعریف فراخوانی می‌شود. برای دیدن این ترتیب، رفتار `Drop` را برای یک نوع ساده تعریف می‌کنیم (نحو `impl` و `for ...` در درس ۳۶ توضیح داده می‌شود):

```
struct Noisy(String);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("آزاد شد: {}", self.0);
    }
}

fn main() {
    let _first = Noisy(String::from("اول"));
    let _second = Noisy(String::from("دوم"));
    println!("پایان main");
}
// خروجی:
// پایان main
// آزاد شد: دوم
// آزاد شد: اول
```

اشتباه رایج: تصور اینکه `clone()` مثل کپی ضمنی همیشه ارزان است؛ کپی‌کردن یک `String` یا `Vec` بزرگ می‌تواند هزینه‌ی حافظه و زمان داشته باشد.

درس ۱۸: Reference و Borrowing

T& و mut T&

با `&expr` یک reference از نوع `&T` می‌سازیم؛ یعنی به مقدار دسترسی داریم، بدون اینکه مالکیتش را بگیریم. به این کار borrowing (قرض‌گرفتن) می‌گویند. با `&mut expr` یک reference قابل‌تغییر از نوع `&mut T` می‌سازیم. برای دسترسی به مقدارِ پشت reference از `*` (dereference) استفاده می‌شود.

```
fn main() {
    let mut value = 10;

    let read: &i32 = &value;
    println!("{read}");

    // قابل‌تغییر reference
    let write: &mut i32 = &mut value;
    *write = 20;

    println!("{value}");
}
```

قرض‌گرفتن در تابع‌ها نیز به همین شکل است و مالکیت را منتقل نمی‌کند:

```rust
fn length(text: &String) -> usize {
    text.len()
}

fn main() {
    let s = String::from("Rust");
    println!("{}", length(&s)); // s قرض داده می‌شود
    println!("{s}");           // s همچنان معتبر است
}
```

قانون reference ها

در هر لحظه، برای یک مقدار یا **یک** `&mut` مجاز است یا **چند** `&`؛ هر دو هم‌زمان مجاز نیست. این محدودیت از تغییر هم‌زمان و data race جلوگیری می‌کند.

```rust
fn main() {
    let mut text = String::from("Rust");

    let a = &text;
    let b = &text; // چند reference خواندنی با هم مجازند
    println!("{a} {b}");

    // پس از آخرین استفاده از a و b، گرفتن &mut ممکن است
    let c = &mut text;
    c.push('!');
    println!("{c}");
}
```

این رفتار به NLL (non-lexical lifetimes) مربوط است: کامپایلر دامنه‌ی استفاده از reference را تا آخرین استفاده‌ی آن در نظر می‌گیرد، نه تا انتهای بلوک.

جلوگیری از dangling reference

کامپایلر اجازه نمی‌دهد reference به مقداری اشاره کند که پیش از استفاده از reference از بین می‌رود:

```
fn main() {
    let r;
    {
        let x = 10;
        r = &x; // x` does not live long enough` :خطای کامپایل
    }
    // در اینجا دیگر وجود ندارد println!("{r}"); // x
}
```

> **اشتباه رایج:** نگه‌داشتن `&mut T` هم‌زمان با reference های دیگر به همان مقدار، به‌ویژه وقتی قرار است مقدار تغییر کند.

درس ۱۹: &str، String و literal های متنی

انواع literal متنی

- `"..."` : رشته‌ی معمولی از نوع `&str` .
- `r"..."` و `r#"..."#` : رشته‌ی خام (raw string)؛ escape ها تفسیر نمی‌شوند.
- `b"..."` : آرایه‌ای از بایت‌ها، برای داده‌های ASCII.

- `c"..."`: رشته‌ی C با پایان‌دهنده‌ی صفر (`\0`) و نوع `&CStr`.
- `'A'`: یک `char`؛ در Rust یک مقدار Unicode است، نه یک بایت.

```
use std::ffi::CStr;

fn main() {
    let a = "سلام\nRust";          // یعنی خط جدید n\
    let b = r#"مسیر: "C:\temp""#;  // raw string: escape ها تفسیر نمی‌شوند
    let c = b"Hello";              // byte string: آرایه‌ای از بایت‌ها
    let d: &CStr = c"Hello";       // C string با پایان‌دهنده‌ی صفر
    let e = 'ش';                   // یک char

    println!("{a}\n{b}\n{c:?}\n{d:?}\n{e}");
}
```

تفاوت String و &str

`String` رشته‌ای قابل‌تغییر است که مالک داده‌اش است؛ `&str` ارجاعی به متن UTF-8 است. با `push_str` می‌توان به یک `String` متن اضافه کرد.

```
fn main() {
    let mut owned = String::from("سلام");
    let borrowed: &str = " دنیا";

    owned.push_str(borrowed);
    println!("{owned}");
}
```

برش رشته و UTF-8

اندیس‌های برش رشته برحسب **بایت** هستند، نه تعداد کاراکتر، و باید روی مرز معتبر UTF-8 باشند.

```
fn main() {
    let text = "Hello";

    // برش معتبر
    println!("{}", &text[0..2]); // He

    let persian = "سلام";
    // println!("{}", &persian[0..1]); // panic :برش وسط یک کاراکتر
    println!("{}", &persian[0..2]);    // «س» که ۲ بایت است
}
```

> **اشتباه رایج:** فرض اینکه هر کاراکتر یک بایت است؛ کاراکترهای فارسی معمولاً دو بایت دارند و برش وسط آن‌ها باعث `panic` می‌شود.

درس ۲۰: ماکروهای رایج و فرمت‌بندی

شکل فراخوانی و ماکروهای چاپ

ماکروها با `!` و یکی از سه نوع جداکننده‌ی `(...)`، `[...]` یا `{...}` فراخوانی می‌شوند. هر ماکرو را می‌توان با هر سه شکل نوشت، اما قرارداد رایج، `println!(...)` و `format!(...)` با پرانتز و `vec![...]` با کروشه است. در این درس فقط **استفاده** از ماکروها را می‌بینید؛ نوشتن ماکرو در درس ۵۸ است.

```rust
fn main() {
    let name = "Rust";
    let n = 12;

    // چاپ در خروجی استاندارد و خروجی خطا
    println!("Hello, {name}!");
    print!("{n} "); // بدون رفتن به خط بعد
    eprintln!("پیام خطا");

    // ساخت رشته و بردار
    let message = format!("{name}: {n}");
    let values = vec![1, 2, 3];
    println!("{message} {values:?}");

    // بررسی شرط‌ها
    assert!(n > 0);
    assert_eq!(n, 12);
}
```

فرمت‌بندی خروجی

- `{}`: نمایش معمولی (Display)
- `{:?}`: نمایش Debug
- `{:#?}`: نمایش Debug چندخطی و خوانا
- `{:>8}`: راست‌چین در عرض ۸
- `{:.2}`: دو رقم اعشار

- `{name}` : استفاده‌ی مستقیم از متغیر (inline argument)

```
fn main() {
    let name = "Ali";
    let price = 12.3456;

    println!("{name:?}");
    println!("{name:#?}");
    println!("[{name:>8}]");
    println!("{price:.2}");
}
```

بررسی و اشکال‌زدایی

```
fn main() {
    let value = Some(5);

    // بررسی تطبیق با الگو
    assert!(matches!(value, Some(5)));

    // چاپ مقدار همراه با محل فراخوانی؛ همان مقدار را برمی‌گرداند
    let n = dbg!(2 + 3);
    println!("{n}");

    // برای بخش‌های هنوز پیاده‌سازی‌نشده یا غیرقابل‌دسترس:
    // todo!("بعداً پیاده‌سازی شود");
    // unimplemented!("هنوز آماده نیست");
    // unreachable!("نباید به اینجا برسیم");
}
```

`todo!`، `unimplemented!` و `unreachable!` هنگام اجرا `panic` می‌کنند. `dbg!` مقدار را همراه با نام فایل و شماره‌ی خط در خروجی خطا چاپ می‌کند و همان مقدار را برمی‌گرداند.

> **اشتباه رایج:** استفاده از `{}` برای نوعی که `Display` ندارد؛ برای بسیاری از ساختارها باید از `{:?}` استفاده کرد.

بخش سوم: مدل‌سازی داده و ساختار برنامه

درس ۲۱: Struct

struct با فیلدهای نام‌دار

`struct` یک نوع جدید می‌سازد. برای ساخت نمونه از `Type { ... }` و برای دسترسی به فیلد از `expr.ident` استفاده می‌کنیم.

```
struct User {
    name: String,
    age: u8,
}

fn main() {
    let name = String::from("Ali");

    let user = User {
        name, // name: name معادل ؛field init shorthand
        age: 30,
    };

    println!("{} {}", user.name, user.age);
}
```

Tuple struct و unit struct

`tuple struct` فیلدهای بدون نام دارد و با `Type(...)` ساخته می‌شود. `unit struct` هیچ فیلدی ندارد.

```
struct Point(i32, i32);
struct Marker;

fn main() {
    let p = Point(10, 20);
    let _m = Marker;

    // دسترسی به فیلدهای tuple struct با اندیس
    println!("{}, {}", p.0, p.1);
}
```

Struct update syntax

با `..base` می‌توان فیلدهای باقی‌مانده را از نمونه‌ای دیگر گرفت. فیلدهایی که صریحاً مشخص شده‌اند، جای مقدارهای `base` را می‌گیرند.

```
struct Config {
    width: u32,
    height: u32,
    enabled: bool,
}

fn main() {
    let base = Config {
        width: 800,
        height: 600,
        enabled: true,
    };

    let small = Config {
        width: 400,
        ..base
    };

    println!("{} {} {}", small.width, small.height, small.enabled);
}
```

اگر فیلدی که از `base` گرفته می‌شود از نوع `Copy` نباشد (مثلاً `String`)، آن فیلد از `base` move می‌شود و `base` پس از آن به‌طور کامل قابل استفاده نیست.

> **اشتباه رایج:** در struct نام‌دار نمی‌توان با اندیس به فیلد دسترسی داشت (`user.0` معتبر نیست)؛ فقط در tuple struct از اندیس استفاده می‌شود.

درس ۲۲: impl و متدها

impl و تابع‌های وابسته به نوع

با `impl Type { ... }` می‌توان تابع‌ها و متدهای مرتبط با یک نوع تعریف کرد. تابعی مثل `new` که پارامتر `self` ندارد، associated function است و با `Type::new()` فراخوانی می‌شود. `Self` به نوعی که `impl` برای آن نوشته شده اشاره می‌کند.

```
struct Point {
    x: i32,
    y: i32,
}

impl Point {
    // associated function یک سازنده؛
    fn new(x: i32, y: i32) -> Self {
        Self { x, y }
    }

    // متد فقط‌خواندنی
    fn sum(&self) -> i32 {
        self.x + self.y
    }
}

fn main() {
    let p = Point::new(3, 4);
    println!("{}", p.sum());
}
```

متدهای تغییردهنده و مصرف‌کننده

- `&self` : دسترسی فقط‌خواندنی به نمونه.
- `&mut self` : امکان تغییر نمونه.
- `self` و `mut self` : مالکیت نمونه را می‌گیرد و آن را مصرف می‌کند.

```
struct Counter {
    value: i32,
}

impl Counter {
    fn increment(&mut self) {
        self.value += 1;
    }

    fn reset(mut self) -> Self {
        self.value = 0;
        self
    }
}

fn main() {
    let mut c = Counter { value: 5 };
    c.increment();
    println!("{}", c.value); // 6

    let c = c.reset(); // c قبلی مصرف شد
    println!("{}", c.value); // 0
}
```

چند بلوک impl و فراخوانی متد

برای یک نوع می‌توان چند بلوک `impl` نوشت. هنگام فراخوانی متد با نقطه، Rust به‌طور خودکار `&`، `&mut` یا `*` لازم را اضافه می‌کند (auto-referencing).

```
struct User {
    name: String,
}

impl User {
    fn rename(&mut self, name: &str) {
        self.name = name.to_string();
    }
}

impl User {
    fn show(&self) {
        println!("{}", self.name);
    }
}

fn main() {
    let mut user = User { name: "Ali".into() };
    user.rename("Sara");
    user.show();
}
```

اشتباه رایج: فراخوانی `new()` با نقطه (`p.new()`)؛ چون `new` به نمونه نیاز ندارد، باید به شکل `Type::new()` فراخوانی شود.

درس ۲۳: Enum

تعریف enum و انواع variant

`enum` نوعی می‌سازد که مقدارش یکی از چند حالت (variant) است؛ به این ساختار sum type می‌گویند. هر variant می‌تواند بدون داده، دارای داده‌ی ترتیبی (tuple) یا دارای فیلدهای نام‌دار باشد.

```
enum Message {
    Quit,                   // بدون داده
    Move(i32, i32),         // داده‌های tuple
    Write { text: String }, // فیلد نام‌دار
}

fn main() {
    let a = Message::Quit;
    let b = Message::Move(10, 20);
    let c = Message::Write {
        text: String::from("Hello"),
    };

    // :: با variant مسیر
    let _ = (a, b, c);
}
```

impl برای enum

مانند struct، با `impl` می‌توان برای `enum` هم متد تعریف کرد.

```
enum Direction {
    North,
    South,
}

impl Direction {
    fn is_north(&self) -> bool {
        matches!(self, Direction::North)
    }
}

fn main() {
    let direction = Direction::North;
    println!("{}", direction.is_north());
    let _ = Direction::South;
}
```

مقدار discriminant و تبدیل با as

برای enum ساده (بدون داده) می‌توان مقدار عددی هر variant را با `as` گرفت. مقدارها به‌طور پیش‌فرض از صفر شروع می‌شوند و می‌توان آن‌ها را صریحاً تعیین کرد.

```
enum Color {
    Red = 1,
    Green = 2,
    Blue = 3,
}

fn main() {
    let color = Color::Green;

    // تبدیل به عدد discriminant
    println!("{}", color as i32); // 2
    let _ = (Color::Red, Color::Blue);
}
```

اشتباه رایج: استفاده از `as` برای تبدیل enum دارای داده به عدد؛ این تبدیل فقط برای enum های بدون داده مجاز است.

درس ۲۴: match

ساختار match و exhaustiveness

`match` یک مقدار را با چند الگو (pattern) مقایسه می‌کند. علامت `=>` الگو را به نتیجه وصل می‌کند. همه‌ی حالت‌های ممکن باید پوشش داده شوند (exhaustiveness)؛ در غیر این صورت کامپایلر خطا می‌دهد.

```
fn main() {
    let number = 2;

    let text = match number {
        1 => "یک",
        2 => "دو",
        _ => "سایر", // پوشش همه‌ی حالت‌های باقی‌مانده
    };

    println!("{text}");
}
```

Arm با بلوک و match guard

هر arm می‌تواند یک بلوک باشد و مقدار آخر بلوک، نتیجه‌ی آن است. با `if` می‌توان شرط اضافه‌ای به الگو داد (match guard).

```
fn main() {
    let number = 7;

    let result = match number {
        n if n % 2 == 0 => {
            // این بلوک یک String تولید می‌کند
            format!("{n} است زوج")
        }
        n => format!("{n} است فرد"),
    };

    println!("{result}");
}
```

اتصال مقدار با @

علامت `@` هم‌زمان مقدار را به یک نام وصل می‌کند و بررسی می‌کند که با الگو مطابقت داشته باشد.

```
fn main() {
    let number = 4;

    match number {
        n @ 1..=5 => println!("{n} است ۵ تا ۱ در بازه‌ی"),
        _ => println!("خارج از بازه"),
    }
}
```

اشتباه رایج: فراموش‌کردن پوشش همه‌ی حالت‌ها؛ اگر الگوها کامل نباشند و حالت جایگزینی مثل `_` هم وجود نداشته باشد، کد کامپایل نمی‌شود.

درس ۲۵: Pattern ها (کامل)

Pattern برای تطبیق مقدار و استخراج بخش‌های آن به کار می‌رود. pattern ها فقط در `match` نیستند؛ در `let`، پارامتر تابع و `for` هم کاربرد دارند.

الگوهای پایه و ترکیبی

- `42`، `"hi"`: literal pattern؛ `name`: اتصال مقدار به یک متغیر (identifier pattern).
- `_`: نادیده‌گرفتن مقدار (wildcard)؛ `..`: کنار گذاشتن بقیه‌ی اجزا (rest pattern).
- `1..=5`، `1..`، `..=5`: range pattern.
- `A | B`: پذیرفتن یکی از چند الگو (or-pattern)؛ `(p)`: گروه‌بندی.
- `CONST` و `Enum::Variant`: path pattern.

```
const LIMIT: i32 = 10;

enum Color {
    Red,
    Blue,
}

fn describe(n: i32) {
    match n {
        0 | 1 => println!("صفر یا یک"),   // or-pattern
        2..=5 => println!("از ۲ تا ۵"),   // range pattern
        (6 | 7) => println!("شش یا هفت"), // گروه‌بندی‌شده pattern
        LIMIT => println!("حد مجاز"),     // path pattern
        _ => println!("سایر"),            // wildcard
    }
}

fn paint(c: Color) {
    match c {
        Color::Red => println!("قرمز"),
        Color::Blue => println!("آبی"),
    }
}

fn main() {
    let (x, _) = (7, 9); // tuple pattern
    describe(x);
    describe(10);
```

```
    paint(Color::Red);
    paint(Color::Blue);
}
```

```
struct Point {
    x: i32,
    y: i32,
}
struct Pair(i32, i32);

fn main() {
    let Point { x, y: _ } = Point { x: 3, y: 4 }; // struct pattern
    let Pair(a, ..) = Pair(8, 9);                 // tuple struct pattern
    let (first, .., last) = (1, 2, 3, 4);         // rest با tuple pattern
    println!("{x} {a} {first} {last}");

    let values = [10, 20, 30];
    if let [head, .., tail] = values {            // slice pattern
        println!("{head} {tail}");
    }

    let value = 5;
    let reference = &value;
    let &n = reference;                           // reference pattern
    println!("{n}");
}
```

mut، ref و pattern در پارامتر و for

`mut` متغیر استخراج‌شده را قابل‌تغییر می‌کند. `ref` و `ref mut` به‌جای moveکردن مقدار، reference به آن می‌گیرند.

```rust
fn print_pair((a, b): (i32, i32)) { // در پارامتر تابع pattern
    println!("{a} {b}");
}

fn main() {
    let (mut count, step) = (0, 2); // pattern در mut
    count += step;
    print_pair((count, step));

    for &item in &[10, 20] {        // for در pattern
        println!("{item}");
    }

    let name = Some(String::from("Ali"));
    if let Some(ref inner) = name { // نمی‌کند move را name و می‌گیرد ref: reference
        println!("{inner}");
    }
    println!("{name:?}");           // همچنان معتبر است name

    let mut v = Some(5);
    if let Some(ref mut n) = v {    // قابل‌تغییر ref mut: reference
        *n += 1;
    }
    println!("{v:?}");              // Some(6)
}
```

در edition 2024، وقتی یک مقدار از طریق reference تطبیق داده می‌شود (مثلاً `match &opt { Some(x) => ... }`)، کامپایلر خودش `x` را به‌صورت reference می‌گیرد. در این حالت نوشتن صریح `ref`، `ref mut`، `mut` یا `&` درون همان الگو خطاست؛ الگو را ساده بنویسید.

> **اشتباه رایج:** اشتباه گرفتن `..` با `..=`؛ در یک expression یا pattern، `..` انتهای بازه را شامل نمی‌شود و `..=` شامل می‌شود. همچنین `..` به‌تنهایی در pattern یعنی «بقیه‌ی اجزا را نادیده بگیر».

درس ۲۶: let else، while let، if let و matches!

if let و matches!

وقتی فقط یک یا چند الگوی خاص برایمان مهم است، `if let` از `match` کوتاه‌تر است و لازم نیست همه‌ی حالت‌ها را بنویسیم. با `|` می‌توان چند الگو را ترکیب کرد.

```
fn main() {
    let value = Some(10);

    if let Some(n) = value {
        println!("مقدار: {n}");
    }

    if let Some(1 | 2) = Some(2) {
        println!("یک یا دو");
    }

    // بررسی تطبیق بدون استخراج مقدار
    println!("{}", matches!(value, Some(1..=10)));
}
```

let else

وقتی تطبیق موفق نباشد، `let else` مسیر جایگزین را اجرا می‌کند. بلوک `else` باید با خروج از مسیر جاری تمام شود؛ مثلاً با `return`، `break`، `continue` یا `panic!`.

```
fn get_number(value: Option<i32>) -> i32 {
    let Some(n) = value else {
        return 0; // در صورت نبود مقدار، تابع همین‌جا تمام می‌شود
    };

    n
}

fn main() {
    println!("{}", get_number(Some(7)));
    println!("{}", get_number(None));
}
```

while let

تا وقتی مقدار با الگو تطبیق دارد، حلقه اجرا می‌شود. این الگو برای پردازش پیاپی `Option` یا `Result` کاربرد دارد؛ مثلاً خالی‌کردن یک پشته:

```
fn main() {
    let mut stack = vec![1, 2, 3];

    // تا وقتی عنصر هست Some(...) و در پایان None برمی‌گرداند pop()
    while let Some(top) = stack.pop() {
        println!("{top}"); // 1 ،2 ،3
    }
}
```

اشتباه رایج: استفاده از `if let` به‌جای `let else` وقتی لازم است در صورت شکست تطبیق، اجرای تابع حتماً متوقف شود؛ `if let` مسیر شکست را به‌طور صریح مجبور به خروج نمی‌کند.

درس ۲۷: Option<T>

Some و None

`Option<T>` نوعی برای نمایش وجود یا نبود مقدار است؛ `<T>` یعنی «هر نوعی» (generic ها در درس ۳۵ بررسی می‌شوند). به‌جای `null`، مقدار یا `Some(value)` است یا `None`.

```
fn main() {
    let a: Option<i32> = Some(10);
    let b: Option<i32> = None;

    println!("{}", a.unwrap());     // 10
    println!("{}", b.unwrap_or(0)); // 0
    println!("{}", a.is_some());    // true

    // expect هنگام None با پیام مشخص panic می‌کند
    println!("{}", a.expect("مقدار موجود نیست"));
}
```

متدهای تبدیل و پردازش

- `unwrap()`: مقدار را می‌گیرد؛ در `None` باعث panic می‌شود.

- `expect("پیام")`: مانند `unwrap`، با پیام مشخص.
- `unwrap_or(default)`: مقدار پیش‌فرض ثابت.
- `unwrap_or_else(|| ...)`: محاسبه‌ی مقدار پیش‌فرض فقط هنگام `None`.
- `map(f)`: تبدیل مقدار داخل `Some`.
- `and_then(f)`: اجرای تابعی که خودش `Option` برمی‌گرداند.
- `ok_or(error)`: تبدیل `Option` به `Result`.
- `?`: در صورت `None`، تابعی که `Option` برمی‌گرداند همان‌جا `None` برمی‌گرداند.

```
fn double(value: Option<i32>) -> Option<i32> {
    value.map(|n| n * 2)
}

fn positive(value: Option<i32>) -> Option<i32> {
    value.and_then(|n| if n > 0 { Some(n) } else { None })
}

fn add_one(value: Option<i32>) -> Option<i32> {
    let n = value?; // اگر None باشد، تابع همین‌جا None برمی‌گرداند
    Some(n + 1)
}

fn main() {
    println!("{:?}", double(Some(4)));    // Some(8)
    println!("{:?}", positive(Some(-2))); // None
    println!("{:?}", add_one(None));      // None
    println!("{:?}", None::<i32>.ok_or("مقدار موجود نیست"));
    println!("{}", None::<i32>.unwrap_or_else(|| 5));
}
```

اشتباه رایج: استفاده از `unwrap()` بدون اطمینان از وجود مقدار؛ اگر مقدار `None` باشد، برنامه `panic` می‌کند.

درس ۲۸: Result، panic! و عملگر ?

Result و تفاوت خطا با panic!

`Result<T, E>` دو حالت دارد: `Ok(value)` برای موفقیت و `Err(error)` برای خطا. `T` نوع مقدار موفق و `E` نوع خطاست.

- خطای قابل‌بازیابی: با `Err` گزارش می‌شود و برنامه می‌تواند تصمیم بگیرد چه کند.
- `panic!`: اجرای عادی را متوقف می‌کند؛ برای خطاهای غیرمنتظره و باگ‌ها مناسب است.
- `unwrap()` و `expect()` در حالت `Err` باعث `panic!` می‌شوند.

```
fn divide(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        return Err("تقسیم بر صفر".to_string());
    }
    Ok(a / b)
}

fn main() {
    match divide(10, 0) {
        Ok(n) => println!("{n}"),
        Err(e) => println!("خطا: {e}"),
    }

    // برای توقف فوری در شرایط غیرمنتظره:
    // panic!("خطای غیرمنتظره");
}
```

عملگر ? و ارسال خطا به بالا

`?` اگر نتیجه `Ok` باشد مقدار داخل آن را بیرون می‌آورد؛ اگر `Err` باشد، همان خطا را از تابع فعلی برمی‌گرداند. بنابراین نوع خطا باید با نوع بازگشتی تابع سازگار باشد.

```
fn divide(a: i32, b: i32) -> Result<i32, String> {
    if b == 0 {
        return Err("تقسیم بر صفر".to_string());
    }
    Ok(a / b)
}

fn calculate() -> Result<i32, String> {
    let a = divide(12, 3)?; // اگر Err باشد، همین‌جا به فراخواننده برمی‌گردد
    let b = divide(a, 2)?;
    Ok(b)
}

fn main() -> Result<(), String> {
    println!("{}", calculate()?);
    Ok(())
}
```

در Rust می‌توان `main` را به شکل `fn main() -> Result<(), E>` نوشت؛ در این حالت `Ok(())` پایان موفق برنامه است و در صورت `Err`، خطا چاپ می‌شود و برنامه با کد خروج غیرصفر تمام می‌شود.

> **اشتباه رایج:** استفاده از `?` در تابعی که نوع بازگشتی آن اجازه‌ی انتقال همان خطا را نمی‌دهد (مثلاً تابعی که `()` برمی‌گرداند).

درس ۲۹: Module و visibility

تعریف module و ساختار فایل‌ها

با `mod name { ... }` ماژول را داخل همان فایل تعریف می‌کنیم؛ با `mod name;` محتوای آن از فایل جداگانه خوانده می‌شود.

ساختار متداول پروژه:

```
src/
├── main.rs
├── tools.rs
└── models/
    ├── mod.rs
    └── user.rs
```

در Rust 2018 و جدیدتر، به‌جای `models/mod.rs` می‌توان از فایل `models.rs` استفاده کرد و زیرماژول‌ها را در پوشه‌ی `models/` گذاشت.

```
// src/main.rs
mod tools; // محتوای src/tools.rs

fn main() {
    tools::run();
}
```

```
// src/tools.rs
pub fn run() {
    println!("اجرا شد");
}
```

Visibility و دسترسی

اعضای ماژول به‌طور پیش‌فرض private هستند؛ یعنی از بیرون ماژول (و از ماژول والد) قابل دسترسی نیستند. `pub` دسترسی را بازتر می‌کند.

```rust
mod data {
    pub struct User {
        pub name: String,
        age: u8, // خصوصی
    }

    impl User {
        pub fn new(name: String, age: u8) -> Self {
            Self { name, age }
        }
    }

    pub(crate) fn internal() {}        // در کل crate
    pub(super) fn parent_only() {}     // در ماژول والد
    pub(self) fn local_only() {}       // فقط در همین ماژول
    pub(in crate::data) fn scoped() {} // در مسیر مشخص‌شده (باید یکی از اجدادِ فعلی باشد)
}

fn main() {
    let user = data::User::new("Ali".into(), 20);
    println!("{}", user.name);
    data::internal();
}
```

- `pub(crate)`: کل crate؛ `pub(super)`: ماژول والد؛ `pub(self)`: همین ماژول (معادل نبودن `pub`).
- `pub(in path)`: ماژول `path` و همه‌ی زیرماژول‌های آن؛ `path` باید یکی از اجدادِ ماژول فعلی باشد.

- دسترسی به فیلدهای struct جداگانه تعیین می‌شود؛ عمومی‌بودن خود struct کافی نیست.

> **اشتباه رایج:** فراموش‌کردن `pub` برای فیلد یا تابعی که باید از ماژول دیگر قابل دسترسی باشد.

درس ۳۰: Path ها و use

انواع path

در Rust، `::` بخش‌های یک مسیر را از هم جدا می‌کند و مشخص می‌کند به کدام ماژول، نوع یا تابع می‌رسیم.

- `ident::item` : عضوی از یک ماژول.
- `crate::item` : از ریشه‌ی crate جاری.
- `self::item` : از ماژول جاری.
- `super::item` : از ماژول والد.
- `::crate_name::item` : از ریشه‌ی یک crate خارجی.

```
mod parent {
    pub fn hello() {
        println!("سلام");
    }

    pub mod child {
        pub fn run() {
            super::hello();         // تابعی در ماژول والد
            crate::parent::hello(); // مسیر کامل از ریشه
            self::local();          // تابعی در همین ماژول
        }

        fn local() {
            println!("محلی");
        }
    }
}

fn main() {
    parent::child::run();

    // (std اینجا) crate مسیر با :: در ابتدا یعنی از ریشه‌ی یک
    let map: ::std::collections::HashMap<i32, i32> = ::std::collections::HashMap::new();
    println!("{}", map.len());
}
```

use و شکل‌های مختلف آن

`use` مسیر را وارد محدوده‌ی فعلی می‌کند تا لازم نباشد هر بار مسیر کامل را بنویسیم.

```
use std::collections::{HashMap, HashSet}; // وارد کردن چند عضو
use std::fmt::Debug as FmtDebug;          // نام مستعار (alias)

fn show<T: FmtDebug>(x: T) {
    println!("{x:?}");
}

fn main() {
    let map: HashMap<i32, i32> = HashMap::new();
    let set: HashSet<i32> = HashSet::new();
    show(map.len() + set.len());
}
```

- `use path as Alias` : نام مستعار.
- `use module::*` : وارد کردن همه‌ی نام‌های قابل‌دسترسی ماژول (glob import).
- `use module::{a, b}` : وارد کردن چند عضو.
- `pub use` : re-export؛ نام را از مسیر دیگری هم در دسترس قرار می‌دهد.
- `use Trait as _` : trait را وارد می‌کند تا متدهایش فعال شوند، بدون وارد کردن نام آن.
- prelude: مجموعه‌ای از نام‌های رایج (مانند `Option` ، `Result` ، `Vec`) که Rust خودکار وارد محدوده می‌کند.

```rust
use std::fmt::Write as _; // Write متد write_str فعال می‌شود، بدون وارد شدن نام

mod shapes {
    pub mod circle {
        pub fn area(r: f64) -> f64 {
            3.14 * r * r
        }
    }
    // re-export: area از مسیر shapes هم در دسترس است
    pub use self::circle::area;
}

use shapes::*; // glob import

fn main() {
    let mut s = String::new();
    s.write_str("سلام").unwrap();
    println!("{s} {}", area(2.0));
}
```

اشتباه رایج: تصور اینکه `use` دسترسی خصوصی را عمومی می‌کند؛ `use` فقط مسیر را کوتاه می‌کند و جایگزین `pub` نیست.

درس ۳۱: Crate و Cargo

تفاوت package، crate و module

- Package: پروژه‌ای با فایل `Cargo.toml` که یک یا چند crate دارد.
- Crate: واحد کامپایل؛ معمولاً `src/main.rs` برای برنامه و `src/lib.rs` برای کتابخانه.
- Module: برای سازمان‌دهی کد داخل crate؛ با `mod` تعریف می‌شود.

```
# Cargo.toml
[package]
name = "my_app"
version = "0.1.0"
edition = "2024"

[dependencies]
rand = "0.9" # وابستگی از crates.io
```

```rust
// src/main.rs
use rand::Rng;

fn main() {
    // تولید عدد تصادفی بین 1 تا 10
    let n = rand::rng().random_range(1..=10);
    println!("{n}");
}
```

کتابخانه، feature و workspace

```toml
# تعریف feature اختیاری :Cargo.toml
[features]
default = []
extra = []
```

```toml
# Cargo.toml در ریشه‌ی یک workspace
[workspace]
members = ["app", "shared"]
resolver = "3"
```

- `src/lib.rs` : ریشه‌ی crate کتابخانه‌ای؛ با `pub` اعضایی را در دسترس مصرف‌کننده قرار می‌دهد.
- Feature: قابلیت اختیاری crate که در `Cargo.toml` تعریف و هنگام ساخت فعال می‌شود.

- Workspace: چند package مرتبط را با یک تنظیم و یک پوشه‌ی خروجی مشترک مدیریت می‌کند.
- crates.io: مخزن عمومی crate های Rust.
- Edition: نسخه‌ی قواعد زبانی، مانند `2024`؛ با نسخه‌ی کامپایلر یکی نیست.
- Profile: تنظیمات ساخت؛ `dev` برای توسعه و `release` برای انتشار بهینه‌شده.
- `extern crate` : در Rust امروزی معمولاً لازم نیست؛ `use` کافی است.

> **اشتباه رایج:** یکی دانستن package و crate؛ هر package می‌تواند یک crate کتابخانه‌ای و چند crate برنامه‌ی اجرایی داشته باشد.

درس ۳۲: Vec<T>

ساخت و تغییر Vec

`Vec<T>` آرایه‌ای با طول قابل‌تغییر است که عناصرش روی heap نگهداری می‌شوند. `T` نوع عناصر را مشخص می‌کند.

```
fn main() {
    let mut v = Vec::new();
    v.push(30);
    v.push(10);
    v.insert(1, 20);         // [10 ,20 ,30] :1 درج در اندیس
    let removed = v.remove(0); // [10 ,20] می‌شود v ؛30
    let last = v.pop();        // [20] می‌شود v ؛Some(10)

    println!("{v:?} {removed} {last:?}");

    let values = vec![5, 2, 5, 3];
    println!("{:?}", values.get(10)); // panic بدون ،None
    // .می‌شود panic باعث values[10]
}
```

پیمایش و تبدیل به slice

```rust
fn main() {
    let mut v = vec![1, 2, 3];

    // فقط‌خواندنی
    for n in &v {
        println!("{n}");
    }

    // تغییر عناصر از طریق reference قابل‌تغییر
    for n in &mut v {
        *n *= 2;
    }

    let slice: &[i32] = &v[..];
    println!("{slice:?}");
}
```

ظرفیت و مرتب‌سازی

`len()` تعداد عناصر و `capacity()` ظرفیت تخصیص‌یافته را نشان می‌دهد. `with_capacity(n)` فضایی برای دست‌کم `n` عنصر از پیش رزرو می‌کند.

```rust
fn main() {
    let mut v = Vec::with_capacity(8);
    v.extend([4, 2, 4, 1, 3]);

    v.sort();              // مرتب‌سازی صعودی: [1, 2, 3, 4, 4]
    v.dedup();             // حذف تکرارهای مجاور: [1, 2, 3, 4]
    v.retain(|n| *n > 1);  // نگه‌داشتن عناصر بزرگ‌تر از 1: [2, 3, 4]

    println!("{v:?}");
    println!("len={}, capacity={}", v.len(), v.capacity());
}
```

اشتباه رایج: تغییر اندازه‌ی `Vec` هنگام پیمایش آن با `&v` یا `&mut v`؛ ابتدا پیمایش را تمام کنید، سپس عناصر را اضافه یا حذف کنید (کامپایلر چنین کدی را به‌خاطر قوانین borrowing رد می‌کند).

درس ۳۳: مجموعه‌ها (HashMap، HashSet، BTreeMap، BTreeSet، VecDeque)

انتخاب ساختار داده

- `HashMap<K, V>`: نگاشت کلید به مقدار؛ معمولاً سریع، بدون ترتیب مشخص.
- `HashSet<T>`: مجموعه‌ی بدون عنصر تکراری، بدون ترتیب مشخص.
- `BTreeMap<K, V>`: نگاشت مرتب‌شده بر اساس کلید.
- `BTreeSet<T>`: مجموعه‌ی مرتب‌شده و بدون تکرار.

- `VecDeque<T>`: صف دوطرفه؛ افزودن و حذف از ابتدا و انتها کارآمد است.

HashMap و entry

```
use std::collections::HashMap;

fn main() {
    let mut scores = HashMap::new();
    scores.insert("Ali", 10);
    scores.insert("Sara", 20);

    println!("{:?}", scores.get("Ali")); // Some(10)
    scores.remove("Sara");

    // اگر کلید وجود نداشته باشد، مقدار اولیه درج می‌شود
    scores.entry("Reza").or_insert(15);
    *scores.entry("Ali").or_default() += 5;

    for (name, score) in &scores {
        println!("{name}: {score}"); // ترتیب نامشخص است
    }
}
```

مجموعه‌ها و صف دوطرفه

```
use std::collections::{BTreeSet, VecDeque};

fn main() {
    let mut set = BTreeSet::new();
    set.insert(3);
    set.insert(1);
    set.insert(3); // تکراری نادیده گرفته می‌شود
    println!("{set:?}"); // مرتب: {3 ,1}

    let mut queue = VecDeque::new();
    queue.push_back(10);
    queue.push_front(5);
    println!("{:?}", queue.pop_front()); // Some(5)
}
```

Ownership: هنگام `insert`، کلید و مقدار به ساختار داده منتقل می‌شوند، مگر اینکه نوعشان `Copy` باشد. متدهایی مثل `get` معمولاً reference برمی‌گردانند.

> **اشتباه رایج:** فرض‌کردن ترتیب پیمایش `HashMap`؛ اگر ترتیب کلیدها مهم است، از `BTreeMap` استفاده کنید.

درس ۳۴: ورودی/خروجی و محیط

خواندن ورودی از stdin

`read_line` یک خط را همراه با `\n` در انتهای آن در `String` قرار می‌دهد. با `trim()` فاصله‌ها و پایان خط حذف می‌شوند.

```
use std::io;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut input = String::new();

    // خواندن یک خط از ورودی استاندارد
    io::stdin().read_line(&mut input)?;
    println!("ورودی: {}", input.trim());

    Ok(())
}
```

نوع `Box<dyn std::error::Error>` یعنی «هر خطایی»؛ توضیح کامل آن در درس‌های ۴۳ و ۵۰ می‌آید.

خواندن و نوشتن فایل

`fs::read_to_string` کل فایل را می‌خواند؛ `fs::write` فایل را می‌سازد یا محتوای قبلی آن را جایگزین می‌کند. عملگر `?` خطا را به فراخواننده منتقل می‌کند.

```rust
use std::fs;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    fs::write("data.txt", "سلام\nRust")?;
    let text = fs::read_to_string("data.txt")?;
    println!("{text}");
    Ok(())
}
```

آرگومان‌های خط فرمان و پیمایش خط‌به‌خط

`env::args()` آرگومان‌های برنامه را می‌دهد؛ اولین آرگومان معمولاً مسیر خود برنامه است. `BufRead::lines()` فایل را خط‌به‌خط می‌خواند و هر خط می‌تواند خطا داشته باشد.

```rust
use std::{
    env,
    fs::File,
    io::{BufRead, BufReader},
    process,
};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // آرگومان اول بعد از نام برنامه، یا مقدار پیش‌فرض
    let path = env::args().nth(1).unwrap_or("data.txt".into());
    let file = BufReader::new(File::open(path)?);

    for line in file.lines() {
        let line = line?;
        if line.is_empty() {
            // خروج فوری با کد خطا؛ معمولاً بازگرداندن Result بهتر است
            process::exit(1);
        }
        println!("{line}");
    }

    Ok(())
}
```

اشتباه رایج: نادیده‌گرفتن خطاهای ورودی/خروجی؛ با `?` اجازه دهید خطا به‌درستی منتقل شود، مگر اینکه بخواهید مشخصاً آن را مدیریت کنید.

بخش چهارم: Generic، Trait و Lifetime

درس ۳۵: Generics

تابع fn f<T>(...)<generic:>

`<T>` یک پارامتر نوع است؛ یعنی تابع می‌تواند با نوع‌های مختلف کار کند، بدون اینکه برای هر نوع تابعی جداگانه بنویسیم.

```
// می‌تواند هر نوعی باشد T
fn identity<T>(value: T) -> T {
    value
}

fn main() {
    let a = identity(10);     // استنباط می‌شود i32 نوع
    let b = identity("Rust"); // استنباط می‌شود str& نوع

    // (<>::) turbofish تعیین صریح نوع با
    let c = identity::<u8>(20);

    println!("{a} {b} {c}");
}
```

```
struct Pair<T> {
    first: T,
    second: T,
}

enum Slot<T> {
    Value(T),
    Empty,
}

impl<T> Pair<T> {
    fn first(&self) -> &T {
        &self.first
    }
}

fn main() {
    let p = Pair { first: 1, second: 2 };
    println!("{} {}", p.first(), p.second);

    let a: Slot<String> = Slot::Value("سلام".into());
    let b: Slot<u8> = Slot::Empty;
    let _ = (a, b);
}
```

نوع‌های تودرتو و استنباط نوع

`Vec<u8>` یعنی برداری از بایت‌ها و `Vec<Vec<u8>>` یعنی برداری از بردارها. `>>` برای بستن دو پارامتر تودرتو معتبر است. علامت `_` به کامپایلر اجازه می‌دهد نوع را خودش استنباط کند.

```
fn main() {
    let data: Vec<Vec<u8>> = vec![vec![1, 2], vec![3, 4]];

    // کامپایلر نوع عنصر را استنباط می‌کند
    let numbers: Vec<_> = vec![10_i32, 20, 30];

    println!("{data:?} {numbers:?}");
}
```

Monomorphization: کامپایلر برای هر نوعی که واقعاً استفاده می‌شود، نسخه‌ی تخصصی کد تولید می‌کند؛ بنابراین generic ها معمولاً هزینه‌ی اضافه‌ای در زمان اجرا ندارند.

> **اشتباه رایج:** تصور اینکه در `Pair<T>` دو فیلد می‌توانند دو نوع متفاوت باشند؛ هر دو باید از یک `T` باشند. برای دو نوع متفاوت باید `Pair<T, U>` تعریف کرد.

درس ۳۶: Trait

تعریف trait و پیاده‌سازی آن

`trait` رفتاری مشترک را تعریف می‌کند؛ مشابه interface در زبان‌هایی مثل Java و C#، اما قواعد پیاده‌سازی و ترکیب آن متفاوت است.

```
trait Speak {
    fn speak(&self);

    // متد پیش‌فرض؛ پیاده‌سازی آن اختیاری است
    fn greet(&self) {
        println!("سلام!");
    }
}

struct Dog;

impl Speak for Dog {
    fn speak(&self) {
        println!("Woof!");
    }
}

fn main() {
    let dog = Dog;
    dog.speak();
    dog.greet();
}
```

فراخوانی متد و در دسترس بودن trait

برای فراخوانی متد trait با نقطه، آن trait باید در محدوده (scope) باشد. در صورت نیاز می‌توان متد را با نام trait هم صریحاً فراخوانی کرد.

```rust
mod animals {
    pub trait Speak {
        fn speak(&self);
    }

    pub struct Cat;

    impl Speak for Cat {
        fn speak(&self) {
            println!("Meow!");
        }
    }
}

fn main() {
    use animals::{Cat, Speak}; // کار نمی‌کند ()Speak، cat.speak بدون وارد کردن

    let cat = Cat;
    cat.speak();

    // trait فراخوانی صریح از طریق
    Speak::speak(&cat);
}
```

قانون orphan

برای پیاده‌سازی یک trait برای یک نوع، باید **یا trait یا نوع** متعلق به crate خودتان باشد. این قانون (orphan rule یا coherence) از تعارض پیاده‌سازی‌های crate های مختلف جلوگیری می‌کند.

```rust
use std::fmt::Display;

trait Describe {
    fn describe(&self) -> &'static str;
}

struct Book;

// مجاز: trait (Describe) متعلق به crate ماست
impl Describe for Book {
    fn describe(&self) -> &'static str {
        "کتاب"
    }
}

// مجاز: نوع (Book) متعلق به crate ماست
impl Display for Book {
    fn fmt(&self, f: &mut std::fmt::Formatter) -> std::fmt::Result {
        write!(f, "Book")
    }
}

// خطای کامپایل: هم Display و هم Vec<i32> از crate دیگری هستند
// impl Display for Vec<i32> { ... }

fn main() {
```

```
    println!("{} {}", Book.describe(), Book);
}
```

اشتباه رایج: تصور اینکه می‌توان هر trait خارجی را برای هر نوع خارجی پیاده‌سازی کرد؛ قانون orphan چنین کاری را ممنوع می‌کند.

درس ۳۷: Trait bound ها

چند trait در یک bound

با `T: TraitA + TraitB` مشخص می‌کنیم نوع `T` باید هر دو trait را پیاده‌سازی کرده باشد. `where` شرط‌های پیچیده را خواناتر می‌کند.

```rust
use std::fmt::{Debug, Display};

// نوع باید هم Display و هم Debug داشته باشد
fn show<T: Display + Debug>(value: T) {
    println!("{value} | {value:?}");
}

fn compare<T, U>(a: T, b: U)
where
    T: Display,
    U: Display + Debug,
{
    println!("{a} {b:?}");
}

fn main() {
    show(42);
    compare("Rust", 10);
}
```

impl Trait در برابر <T: Trait>

هر دو شکل یک شرط trait اعمال می‌کنند؛ اما `impl Trait` در پارامتر، شکل کوتاه‌تری برای یک نوعِ بی‌نام است. با `<T: Trait>` می‌توان همان نوع را در چند جای امضا به کار برد.

```rust
use std::fmt::Display;

// نوع ورودی نام ندارد
fn print_one(value: impl Display) {
    println!("{value}");
}

// هر دو ورودی باید دقیقاً یک نوع باشند
fn print_two<T: Display>(a: T, b: T) {
    println!("{a} {b}");
}

fn main() {
    print_one("سلام");
    print_two(10, 20);
    // print_two(10, "متن"); // خطای کامپایل: نوع‌ها یکسان نیستند
}
```

شرط روی متدهای یک نوع generic

می‌توان متدهایی را فقط برای نوع‌هایی در دسترس گذاشت که شرط خاصی را برآورده می‌کنند.

```rust
use std::fmt::Display;

struct Boxed<T>(T);

impl<T> Boxed<T> {
    fn new(value: T) -> Self {
        Self(value)
    }
}

// این متد فقط وقتی T قابل نمایش باشد وجود دارد
impl<T: Display> Boxed<T> {
    fn print(&self) {
        println!("{}", self.0);
    }
}

fn main() {
    Boxed::new(25).print();
}
```

اشتباه رایج: تصور اینکه `impl Trait` در دو پارامتر الزاماً یک نوع می‌پذیرد؛ هر پارامتر `impl Trait` می‌تواند نوع مستقل خودش را داشته باشد.

درس ۳۸: Attribute ها (مقدماتی) و derive

شکل attribute ها و derive

Attribute به‌شکل `#[meta]` به آیتم بعد از خود (مثل تابع یا struct) اعمال می‌شود. شکل `#![meta]` به آیتمی که آن را در بر گرفته، معمولاً کل فایل، اعمال می‌شود.

`#[derive(...)]` پیاده‌سازی خودکار trait های رایج را درخواست می‌کند:

- `Debug` : نمایش برای اشکال‌زدایی؛ `Clone` : کپی صریح؛ `Copy` : کپی ضمنی.
- `PartialEq` و `Eq` : مقایسه‌ی برابری؛ `PartialOrd` و `Ord` : مقایسه و ترتیب.
- `Hash` : تولید مقدار هش؛ `Default` : مقدار پیش‌فرض.

```
// این attribute روی struct اعمال می‌شود
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point::default();
    let q = p; // به‌دلیل Copy، p همچنان قابل استفاده است
    println!("{p:?} {}", p == q);
}
```

کنترل هشدارها

- `#[allow(...)]` : نادیده‌گرفتن هشدار مشخص.
- `#[warn(...)]` : نمایش هشدار.
- `#[deny(...)]` : تبدیل هشدار مشخص به خطای کامپایل.

```
#![warn(unused_variables)] // برای کل فایل

#[allow(unused_variables)]
fn example() {
    let x = 10; // هشدار مربوط به این متغیر نادیده گرفته می‌شود
}

fn main() {
    example();
}
```

attribute های پیشرفته‌تر مثل `cfg` و `repr` در درس‌های ۵۷ و ۶۴ می‌آیند.

پیاده‌سازی دستی Debug و Display

`Debug` برای توسعه و بررسی داخلی است؛ `Display` برای نمایش خوانا به کاربر. `Display` را باید دستی پیاده‌سازی کرد. (نشانه‌ی `'_` در `Formatter<'_>` یک lifetime بی‌نام است که در درس ۴۶ توضیح داده می‌شود.)

```rust
use std::fmt;

struct User(String);

impl fmt::Display for User {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "کاربر: {}", self.0)
    }
}

impl fmt::Debug for User {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.debug_tuple("User").field(&self.0).finish()
    }
}

fn main() {
    let user = User("Ali".into());
    println!("{user} | {user:?}");
}
```

اشتباه رایج: تصور اینکه `derive(Debug)` به‌طور خودکار `Display` را هم پیاده‌سازی می‌کند.

درس ۳۹: Operator overloading

trait های std::ops

در Rust می‌توان رفتار عملگرها را برای نوع‌های خودمان تعریف کرد. این کار با پیاده‌سازی trait های مشخص انجام می‌شود:

- `Add`, `Sub`, `Mul`: عملگرهای `+`، `-` و `*`
- `Neg`, `Not`: عملگرهای یکانی `-` و `!`
- `Index`, `IndexMut`: دسترسی با `[]` و دسترسی قابل‌تغییر
- `AddAssign`: عملگر `+=`
- `PartialEq`: عملگرهای `==` و `!=`
- `PartialOrd`: عملگرهای `<`، `>`، `<=` و `>=`

پیاده‌سازی Add برای بردار دوبعدی

```rust
use std::ops::Add;

#[derive(Debug, PartialEq)]
struct Vec2 {
    x: i32,
    y: i32,
}

impl Add for Vec2 {
    type Output = Vec2;

    fn add(self, other: Vec2) -> Vec2 {
        // مؤلفه‌های دو بردار را جمع می‌کنیم
        Vec2 {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}

fn main() {
    let a = Vec2 { x: 1, y: 2 };
    let b = Vec2 { x: 3, y: 4 };

    let c = a + b;
    println!("{c:?}"); // Vec2 { x: 4, y: 6 }
}
```

`Output` نوع نتیجه‌ی عملگر است. در این مثال، `add` مالکیت هر دو بردار را می‌گیرد.

AddAssign، Neg و Index

```
use std::ops::{AddAssign, Index, Neg};

#[derive(Debug, Clone, Copy)]
struct Vec2 {
    x: i32,
    y: i32,
}

impl Neg for Vec2 {
    type Output = Vec2;
    fn neg(self) -> Vec2 {
        Vec2 { x: -self.x, y: -self.y }
    }
}

impl AddAssign for Vec2 {
    fn add_assign(&mut self, other: Vec2) {
        self.x += other.x;
        self.y += other.y;
    }
}

struct Numbers(Vec<i32>);

impl Index<usize> for Numbers {
    type Output = i32;
    fn index(&self, i: usize) -> &i32 {
        &self.0[i]
```

```
    }
}

fn main() {
    let mut v = Vec2 { x: 1, y: 2 };
    v += Vec2 { x: 10, y: 10 };
    println!("{:?} {:?}", v, -v);

    let n = Numbers(vec![5, 6, 7]);
    println!("{}", n[1]); // 6
}
```

برای `IndexMut` متد `index_mut` پیاده‌سازی می‌شود که `&mut` برمی‌گرداند و `n[1] = 9` را ممکن می‌کند.

برابری و ترتیب

```
#[derive(PartialEq, PartialOrd)]
struct Score(i32);

fn main() {
    let a = Score(10);
    let b = Score(20);

    // این عملگرها از trait های مشتق‌شده استفاده می‌کنند
    println!("{}", a != b);
    println!("{}", a < b);
    println!("{}", a >= b);
}
```

اشتباه رایج: تصور اینکه پیاده‌سازی `Add` خودبه‌خود عملگرهای `+=` یا `-` را هم تعریف می‌کند؛ هرکدام trait جداگانه‌ای دارند.

درس ۴۰: Trait های تبدیل

From و Into

`From<A> for B` تبدیل از `A` به `B` را تعریف می‌کند. با پیاده‌سازی `From`، `Into` هم خودکار فراهم می‌شود.

```rust
#[derive(Debug)]
struct Age(u8);

impl From<u8> for Age {
    fn from(value: u8) -> Self {
        Age(value)
    }
}

fn main() {
    let a = Age::from(20);
    let b: Age = 30u8.into();
    println!("{a:?} {b:?}");
}
```

تبدیل قابل‌خطا و تبدیل رشته

`TryFrom` و `TryInto` برای تبدیل‌هایی هستند که ممکن است شکست بخورند. `FromStr` و `.parse()` متن را به نوع مقصد تبدیل می‌کنند.

```rust
use std::str::FromStr;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let n = i32::from_str("42")?;
    let small = u8::try_from(n)?;

    // می‌گیرد annotation نوع مقصد را از parse
    let value: i32 = "123".parse()?;

    println!("{small} {value}");
    Ok(())
}
```

AsRef، Display، Default و ?

- `AsRef<T>` دسترسی به reference ای از نوع `T` می‌دهد، بدون انتقال مالکیت.
- `ToString` با پیاده‌سازی `Display` به‌طور خودکار فراهم می‌شود.
- `Default` مقدار پیش‌فرض را می‌سازد.
- `?` در تابعی که خطا برمی‌گرداند، خطا را به نوع خطای خروجی تبدیل می‌کند؛ این کار از طریق `From` انجام می‌شود.

```rust
fn show<T: AsRef<str>>(s: T) {
    println!("{}", s.as_ref());
}

fn main() {
    let s = String::from("Rust");
    show(&s);   // است <AsRef<str هم String&
    show("hi"); // هم str&

    let text = 42.to_string(); // Display از ToString
    let n = i32::default();    // صفر
    println!("{text} {n}");
}
```

اشتباه رایج: فرض اینکه `From` برای تبدیل‌های نامطمئن مناسب است؛ اگر تبدیل ممکن است شکست بخورد، معمولاً `TryFrom` انتخاب درست‌تری است.

درس ۴۱: Associated type و associated const

Associated type در trait

`type Item` در یک trait، نوعی را مشخص می‌کند که هر پیاده‌سازی باید تعیین کند. برای نمونه، `Iterator` یک associated type به نام `Item` دارد که نوع مقدارهای تولیدشده را نشان می‌دهد.

```
struct Counter(u32);

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<Self::Item> {
        // تا رسیدن به 3 مقدار تولید می‌کند
        if self.0 < 3 {
            self.0 += 1;
            Some(self.0)
        } else {
            None
        }
    }
}

fn main() {
    for n in Counter(0) {
        println!("{n}");
    }
}
```

تفاوت با generic parameter و تعیین نوع

در `trait Trait<T>`، نوع `T` یک پارامتر generic است و یک نوع می‌تواند آن trait را برای چند `T` مختلف پیاده‌سازی کند. اما برای هر پیاده‌سازی trait دارای associated type، فقط یک نوع مشخص می‌شود.

```
trait Source {
    type Item;
    fn get(&self) -> Self::Item;
}

struct Number;

impl Source for Number {
    type Item = i32;

    fn get(&self) -> Self::Item {
        42
    }
}

// bound (associated type binding) در associated type تعیین
fn print_value<S: Source<Item = i32>>(s: &S) {
    println!("{}", s.get());
}

fn main() {
    print_value(&Number);
}
```

Associated const

یک trait یا یک نوع می‌تواند ثابت مرتبط داشته باشد. برای دسترسی به آن از `Type::CONST` استفاده می‌شود.

```
trait Limits {
    const MAX: u32;
}

struct Config;

impl Limits for Config {
    const MAX: u32 = 100;
}

struct Screen;

impl Screen {
    const WIDTH: u32 = 1920; // خود نوع associated const
}

fn main() {
    println!("{} {}", Config::MAX, Screen::WIDTH);
}
```

اشتباه رایج: نوشتن `Source<i32>` برای تعیین associated type؛ شکل درست آن `Source<Item = i32>` است.

درس ۴۲: رفع ابهام در فراخوانی و supertrait

رفع ابهام در فراخوانی متد

اگر دو trait متدی هم‌نام داشته باشند، با `Trait::method(&x)` مشخص می‌کنیم کدام متد اجرا شود. شکل `<Type as Trait>::method(&x)` صریح‌تر است و هم نوع و هم trait را مشخص می‌کند.

```
trait A {
    fn show(&self);
}
trait B {
    fn show(&self);
}
struct Item;

impl A for Item {
    fn show(&self) {
        println!("A");
    }
}
impl B for Item {
    fn show(&self) {
        println!("B");
    }
}

fn main() {
    let x = Item;
    // x.show() مبهم است و کامپایل نمی‌شود؛ باید انتخاب کنیم:
    A::show(&x);
    B::show(&x);
    <Item as A>::show(&x);
}
```

شکل `<Type>::...` برای نوع‌هایی که نامشان را نمی‌توان مستقیم نوشت یا برای رفع ابهام نحوی به کار می‌رود؛ مثلاً `<[T]>::...` و `<&T>::...`:

```
fn main() {
    let v = <Vec<i32>>::new();          // معادل ()Vec::<i32>::new
    let n = <[i32]>::len(&[1, 2, 3]);   // متد len روی نوع slice
    println!("{} {}", v.len(), n);
}
```

Supertrait

با `trait A: B`، هر نوعی که `A` را پیاده‌سازی می‌کند باید `B` را هم پیاده‌سازی کرده باشد. بنابراین داخل متدهای `A` می‌توان از متدهای `B` استفاده کرد.

```
trait Speak {
    fn speak(&self);
}

// پیاده‌سازی Announce مستلزم پیاده‌سازی Speak است
trait Announce: Speak {
    fn announce(&self) {
        self.speak();
        println!("اعلان شد");
    }
}

struct Dog;

impl Speak for Dog {
    fn speak(&self) {
        println!("هاپ");
    }
}

impl Announce for Dog {}

fn main() {
    Dog.announce();
}
```

Blanket impl

در blanket impl، یک trait را برای همه‌ی نوع‌هایی که شرط معینی دارند، یکجا پیاده‌سازی می‌کنیم.

```rust
trait Name {
    fn name(&self) -> &str;
}

trait Describe {
    fn describe(&self);
}

// برای هر نوعی که Name دارد پیاده‌سازی می‌شود Describe
impl<T: Name> Describe for T {
    fn describe(&self) {
        println!("{}", self.name());
    }
}

struct User;

impl Name for User {
    fn name(&self) -> &str {
        "Ali"
    }
}

fn main() {
    User.describe();
}
```

اشتباه رایج: تصور اینکه `trait A: B` یعنی `A` به‌طور خودکار برای هر نوعی که `B` دارد پیاده‌سازی می‌شود؛ معنای آن فقط این است که برای پیاده‌سازی `A` باید `B` هم پیاده‌سازی شده باشد.

درس ۴۳: impl Trait در خروجی و trait object

impl Trait در خروجی

`fn f() -> impl Trait` یعنی تابع مقداری با یک نوعِ مشخص (که `Trait` را پیاده‌سازی کرده) برمی‌گرداند؛ نوع دقیق آن در امضای تابع نشان داده نمی‌شود. نوع خروجی در هر تابع باید یکسان باشد.

```
use std::fmt::Display;

// نوع دقیق خروجی پنهان است
fn message() -> impl Display {
    42
}

fn main() {
    println!("{}", message());
}
```

dyn Trait و dynamic dispatch

`dyn Trait` امکان کار با مقدارهایی از نوع‌های مختلف را از طریق یک trait مشترک فراهم می‌کند. `&dyn Trait` یک reference و `Box<dyn Trait>` مالکیت یک مقدار را نگه می‌دارد.

```
trait Animal {
    fn sound(&self);
}

struct Cat;
struct Dog;

impl Animal for Cat {
    fn sound(&self) {
        println!("Meow");
    }
}
impl Animal for Dog {
    fn sound(&self) {
        println!("Woof");
    }
}

fn play(a: &dyn Animal) {
    // متد در زمان اجرا از طریق vtable انتخاب می‌شود
    a.sound();
}

fn main() {
    play(&Cat);
    play(&Dog);
}
```

انتخاب روش مناسب

- Static dispatch: با generic و `impl Trait` کامپایلر نوع دقیق را می‌داند؛ امکان inline و بهینه‌سازی بیشتر است.
- Dynamic dispatch: با `dyn Trait` فراخوانی از طریق vtable انجام می‌شود؛ برای مجموعه‌ای از نوع‌های متفاوت (مثلاً `Vec<Box<dyn Trait>>`) مناسب است.
- `Box<dyn Trait>` برای نگهداری مالکانه و `&dyn Trait` برای قرض‌گرفتن به کار می‌رود.
- Object safety: شرط‌هایی که اجازه می‌دهند یک trait پشت `dyn` قرار بگیرد؛ مثلاً متدهای generic معمولاً با `dyn Trait` سازگار نیستند.

```
trait Draw {
    fn draw(&self);
}

struct Circle;

impl Draw for Circle {
    fn draw(&self) {
        println!("Circle");
    }
}

// نگهداری مالکانه‌ی یک شیء با نوع پنهان
fn create() -> Box<dyn Draw> {
    Box::new(Circle)
}

fn main() {
    create().draw();
}
```

اشتباه رایج: یکی دانستن `impl Trait` و `dyn Trait`؛ اولی یک نوع مشخص را پنهان می‌کند، اما دومی امکان انتخاب پیاده‌سازی در زمان اجرا را می‌دهد.

درس ۴۴: Closure و function pointer

ساخت closure و capture متغیرها

Closure تابعی بی‌نام است: `|x| x + 1` یا `|x: i32| -> i32 { x + 1 }`. برای closure بدون پارامتر از `||` استفاده می‌شود. Closure می‌تواند متغیرهای محیط را با reference بگیرد یا با `move` مالکیتشان را بگیرد.

```
fn main() {
    let factor = 3;

    // متغیر بیرونی را با reference می‌خواند
    let multiply = |x| x * factor;
    println!("{}", multiply(4));

    // مقدار بیرونی را با move می‌گیرد
    let text = String::from("Rust");
    let show = move || println!("{text}");
    show();
}
```

سه trait مربوط به closure

- `Fn` : محیط را فقط می‌خواند؛ چندبار قابل فراخوانی است.
- `FnMut` : محیط را تغییر می‌دهد؛ چندبار قابل فراخوانی است.
- `FnOnce` : ممکن است مقداری را از محیط مصرف کند؛ فقط یک‌بار تضمین فراخوانی دارد.

```rust
fn call_twice<F: Fn()>(f: F) {
    f();
    f();
}

fn call_mut<F: FnMut()>(mut f: F) {
    f();
    f();
}

fn call_once<F: FnOnce() -> String>(f: F) -> String {
    f()
}

fn make_adder(n: i32) -> impl Fn(i32) -> i32 {
    move |x| x + n // برگشتی، n را نگه می‌دارد closure
}

fn main() {
    let name = String::from("Rust");
    call_twice(|| println!("{name}")); // فقط می‌خواند :Fn

    let mut count = 0;
    call_mut(|| count += 1);           // تغییر می‌دهد :FnMut
    println!("{count}");                // 2

    let s = String::from("متن");
    let result = call_once(move || s);  // را مصرف می‌کند FnOnce: s
```

```
    println!("{result}");

    let add5 = make_adder(5);
    println!("{}", add5(10)); // 15
}
```

Function pointer

نوع `fn(A) -> B` اشاره‌گری به یک تابع معمولی است؛ برخلاف closure نمی‌تواند محیط را capture کند.

```
fn double(x: i32) -> i32 {
    x * 2
}

fn apply(f: fn(i32) -> i32, n: i32) -> i32 {
    f(n)
}

fn main() {
    // تابع معمولی به function pointer تبدیل می‌شود
    println!("{}", apply(double, 4));

    // closure بدون capture هم می‌تواند تبدیل شود
    println!("{}", apply(|x| x + 1, 4));
}
```

اشتباه رایج: تصور اینکه `move` الزاماً closure را `FnOnce` می‌کند؛ نوع trait به نحوه‌ی استفاده از مقدارهای captureشده بستگی دارد، نه به وجود `move`.

درس ۴۵: Iterator

Iterator و تفاوت روش‌های پیمایش

`Iterator` متدی به نام `next()` دارد که هر بار `Some(value)` و در پایان `None` برمی‌گرداند. حلقه‌ی `for x in expr` به‌طور ساده معادل گرفتن یک iterator و فراخوانی مکرر `next()` تا رسیدن به `None` است.

- `iter()`: عناصر را به‌صورت reference می‌خواند.
- `iter_mut()`: عناصر را به‌صورت reference قابل‌تغییر می‌دهد.
- `into_iter()`: عناصر را بر اساس نوع مجموعه و زمینه، معمولاً با انتقال مالکیت، پیمایش می‌کند.

```rust
fn main() {
    let mut nums = vec![1, 2, 3];

    for x in nums.iter() {
        println!("{x}"); // خواندن
    }
    for x in nums.iter_mut() {
        *x *= 2; // تغییر
    }
    let owned: Vec<i32> = nums.into_iter().collect(); // انتقال مالکیت
    println!("{owned:?}");
}
```

Adapter ها و consumer ها

Adapter هایی مثل `map` ، `filter` ، `enumerate` ، `zip` ، `chain` ، `take` ، `skip` ، `rev` یک iterator جدید می‌سازند و lazy هستند؛ تا مصرف نشوند هیچ کاری نمی‌کنند.
Consumer هایی مثل `collect` ، `sum` ، `fold` ، `count` ، `any` ، `all` پیمایش را واقعاً انجام می‌دهند.

```rust
fn main() {
    let result: Vec<i32> = (1..=6)
        .filter(|x| x % 2 == 0)
        .map(|x| x * 10)
        .collect();

    let sum = result.iter().sum::<i32>();
    println!("{result:?} sum={sum}"); // [20, 40, 60] sum=120
}
```

پیاده‌سازی Iterator برای نوع خودمان

```
struct Counter(u32);

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<Self::Item> {
        // سه عدد تولید می‌کند
        if self.0 < 3 {
            self.0 += 1;
            Some(self.0)
        } else {
            None
        }
    }
}

fn main() {
    let total = Counter(0).fold(0, |sum, n| sum + n);
    println!("{total}"); // 6
}
```

اشتباه رایج: تصور اینکه `map` یا `filter` به‌تنهایی نتیجه را محاسبه می‌کند؛ این adapter ها lazy هستند و به consumer ای مثل `collect` نیاز دارند.

درس ۴۶: Lifetime (پایه)

چرا lifetime لازم است؟

Lifetime به کامپایلر کمک می‌کند مطمئن شود هیچ reference ای بیشتر از عمر داده‌ای که به آن اشاره می‌کند، استفاده نمی‌شود. نماد `'a` نامی برای یک lifetime است؛ عمر داده را زیاد نمی‌کند، بلکه فقط **رابطه‌ی عمر** reference ها را بیان می‌کند.

```
// خروجی به هر دو ورودی وابسته است
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let a = String::from("Rust");
    let b = String::from("Lifetime");
    println!("{}", longest(&a, &b));
}
```

اگر نتیجه بیش از یکی از ورودی‌ها زنده بماند، کامپایلر خطا می‌دهد:

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let a = String::from("Rust");
    let result;
    {
        let b = String::from("Lifetime");
        result = longest(&a, &b);
    } // اینجا از بین میرود b
    // b` does not live long enough` :خطای کامپایل // ;println!("{result}")
    let _ = result;
}
```

Lifetime در struct و impl

اگر یک struct یک reference نگه می‌دارد، باید lifetime آن را مشخص کنیم. در `impl` نیز lifetime به نوع متصل می‌شود.

```
struct Text<'a> {
    value: &'a str,
}

// یعنی _' ای که کامپایلر خودش استنباط می‌کند» lifetime
impl Text<'_> {
    fn show(&self) {
        println!("{}", self.value);
    }
}

fn main() {
    let s = String::from("Hello");
    let t = Text { value: &s };
    t.show();
}
```

Elision و 'static

در بسیاری از توابع لازم نیست lifetime را بنویسید؛ کامپایلر طبق قواعد elision آن را استنباط می‌کند:

1. هر پارامتر reference یک lifetime مستقل می‌گیرد.
2. اگر دقیقاً یک lifetime ورودی وجود داشته باشد، به همه‌ی خروجی‌ها داده می‌شود.
3. اگر متدی `&self` یا `&mut self` داشته باشد، lifetime آن به خروجی‌ها داده می‌شود.

`'static` یعنی reference می‌تواند تمام مدت اجرای برنامه معتبر باشد؛ رشته‌ی literal مانند `"Rust"` چنین lifetime ای دارد.

```rust
fn first(s: &str) -> &str {
    // به‌صورت خودکار استنباط می‌شود lifetime
    &s[..1]
}

fn greeting() -> &'static str {
    "Hello"
}

fn main() {
    println!("{} {}", first("Rust"), greeting());
}
```

> **اشتباه رایج:** تصور اینکه `'a` عمر داده را افزایش می‌دهد؛ فقط رابطه‌ی اعتبار reference ها را برای کامپایلر مشخص می‌کند.

درس ۴۷: Lifetime (پیشرفته)

فرض این درس این است که با lifetime پایه (elision، `'a` در تابع و struct) آشنا هستید.

Lifetime bound ها

- `T: 'a`: نوع `T` نباید شامل reference هایی با عمر کوتاه‌تر از `'a` باشد.
- `'a: 'b`: یعنی `'a` دست‌کم به‌اندازه‌ی `'b` معتبر است.

- `T: Trait + 'a`: نوع `T` هم باید `Trait` را پیاده‌سازی کند و هم شرط lifetime را داشته باشد.
- `T: 'static`: نوع `T` شامل هیچ reference ای با عمر کوتاه‌تر از `'static` نیست؛ لزوماً به معنی «تا پایان برنامه زنده ماندن» نیست.

```
use std::fmt::Display;

// T باید Display باشد و نباید reference کوتاه‌عمرتر از 'a داشته باشد
fn show<'a, T: Display + 'a>(value: &'a T) {
    println!("{value}");
}

// a: 'b یعنی 'a دست‌کم به‌اندازه‌ی 'b زنده است؛ پس برگرداندن long به‌عنوان &'b مجاز است
fn pick<'a: 'b, 'b>(long: &'a str, _short: &'b str) -> &'b str {
    long
}

fn needs_static<T: 'static>(_value: T) {}

fn main() {
    show(&42);
    println!("{}", pick("کوتاه" ,"طولانی"));

    needs_static(String::from("owned")); // مجاز: String هیچ reference ای ندارد
    needs_static(10);
    // let local = String::from("x");
    // needs_static(&local); // خطای کامپایل: local& عمر 'static ندارد
}
```

Higher-ranked trait bound

`for<'a>` یعنی شرط باید برای **هر** lifetime ممکن برقرار باشد، نه فقط یک lifetime مشخص.

```
// تابع f باید با هر reference ای را بپذیرد
fn use_fn<F>(f: F)
where
    F: for<'a> Fn(&'a str) -> &'a str,
{
    println!("{}", f("Rust"));
}

fn main() {
    use_fn(|s| s);
}
```

Variance و use<...>

Variance مشخص می‌کند آیا می‌توان نوعی با یک lifetime را جای نوعی با lifetime دیگر به کار برد. برای مثال، `&'long T` را می‌توان جای `&'short T` استفاده کرد (covariant)، اما `&mut T` نسبت به `T` invariant است؛ یعنی نمی‌توان `&mut &'long str` را به `&mut &'short str` تبدیل کرد.

در edition 2024، نوع `impl Trait` در خروجی تابع به‌طور پیش‌فرض همه‌ی lifetime های در دسترس را capture می‌کند. با `use<...>` می‌توان دقیقاً مشخص کرد چه چیزهایی capture شوند؛ `use<>` یعنی هیچ lifetime ای capture نشود:

```rust
// خروجی به عمر s وابسته نیست، چون فقط n (یک عدد) را capture می‌کند
fn len_later(s: &str) -> impl Fn() -> usize + use<> {
    let n = s.len();
    move || n
}

fn main() {
    let f;
    {
        let text = String::from("Rust");
        f = len_later(&text);
    } // text از بین رفت، اما f همچنان معتبر است
    println!("{}", f());
}
```

اشتباه رایج: برداشت اینکه `T: 'static` یعنی مقدار `T` حتماً تا پایان برنامه زنده می‌ماند؛ این شرط درباره‌ی reference های داخل نوع است، نه طول عمر خود مقدار.

درس ۴۸: Smart pointer ها

Box<T>، Deref و Drop

`Box<T>` مقدار را روی heap نگه می‌دارد و برای نوع‌های بازگشتی مثل لیست پیوندی ضروری است. trait `Deref` اجازه می‌دهد با `*` به مقدار اشاره‌شده دسترسی داشته باشیم و trait `Drop` هنگام پایان عمر مقدار، پاک‌سازی سفارشی انجام می‌دهد.

```rust
// لیست پیوندی بازگشتی با Box
enum List {
    Node(i32, Box<List>),
    End,
}

fn main() {
    let list = List::Node(1, Box::new(List::Node(2, Box::new(List::End))));

    // Box به‌طور خودکار آزاد می‌شود
    if let List::Node(n, _) = list {
        println!("{n}");
    }
}
```

```rust
use std::ops::Deref;

struct MyBox<T>(T);

impl<T> Deref for MyBox<T> {
    type Target = T;
    fn deref(&self) -> &T {
        &self.0
    }
}

impl<T> Drop for MyBox<T> {
    fn drop(&mut self) {
        println!("MyBox آزاد شد");
    }
}

fn hello(name: &str) {
    println!("Hello, {name}");
}

fn main() {
    let b = MyBox(String::from("Rust"));
    println!("{}", *b); // دستی dereference
    hello(&b);           // deref coercion: &MyBox<String> → &String → &str
}
```

اشتراک مالکیت با Rc<T>

`Rc<T>` اجازه می‌دهد چند بخش از برنامه مالک مشترک یک مقدار باشند. با حذف آخرین `Rc`، مقدار آزاد می‌شود. `Weak<T>` یک reference ضعیف است که مالکیت را زیاد نمی‌کند و برای جلوگیری از چرخه‌ی مالکیت (که باعث نشت حافظه می‌شود) به کار می‌رود.

```
use std::rc::{Rc, Weak};

fn main() {
    let a = Rc::new(String::from("Rust"));
    let b = Rc::clone(&a); // شمارنده‌ی مالکیت زیاد می‌شود
    let w: Weak<String> = Rc::downgrade(&a);

    println!("{} {} count={}", b, a, Rc::strong_count(&a)); // count=2
    println!("{}", w.upgrade().is_some());                  // true
}
```

Arc<T> و deref coercion

`Arc<T>` مثل `Rc<T>` است، اما برای اشتراک مالکیت بین thread ها طراحی شده است (درس ۵۴). Deref coercion باعث می‌شود reference هایی مثل `&Box<String>` یا `&MyBox<String>` به‌طور خودکار به `&str` تبدیل شوند:

```
fn print_text(s: &str) {
    println!("{s}");
}

fn main() {
    let value = Box::new(String::from("Hello"));
    // تبدیل خودکار از &Box<String> به &str
    print_text(&value);
}
```

> **اشتباه رایج:** استفاده از `Rc` برای اشتراک بین thread ها؛ برای این کار باید از `Arc` استفاده کرد (`Rc` اصلاً `Send` نیست).

درس ۴۹: Interior mutability

Cell<T> و RefCell<T>

Interior mutability یعنی امکان تغییر یک مقدار از طریق reference غیرقابل‌تغییر، با رعایت قواعد ایمنی.

- `Cell<T>` برای جایگزین‌کردن مقدار مناسب است؛ معمولاً با `get()` و `set()` برای نوع‌های `Copy`.
- `RefCell<T>` بررسی borrowing را از زمان کامپایل به زمان اجرا منتقل می‌کند: `borrow()` برای خواندن و `borrow_mut()` برای تغییر. نقض قواعد در زمان اجرا باعث `panic` می‌شود.

```
use std::cell::{Cell, RefCell};

fn main() {
    let n = Cell::new(10);
    n.set(20); // تغییر بدون نیاز به mut&
    println!("{}", n.get());

    let text = RefCell::new(String::from("Rust"));
    text.borrow_mut().push('!');
    println!("{}", text.borrow());
}
```

الگوی Rc<RefCell<T>>

این ترکیب اجازه می‌دهد چند مالک یک مقدار مشترک داشته باشند و آن را تغییر دهند؛ معمولاً در برنامه‌های تک‌نخی استفاده می‌شود.

```
use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    let a = Rc::new(RefCell::new(5));
    let b = Rc::clone(&a);

    *b.borrow_mut() += 3;
    println!("{}", a.borrow()); // 8
}
```

Cow<'a, T>

`Cow` مخفف Clone on Write است: می‌تواند داده‌ی قرض‌گرفته‌شده را نگه دارد و فقط هنگام نیاز به تغییر، نسخه‌ای مالکانه از آن بسازد.

```
use std::borrow::Cow;

fn normalize(s: &str) -> Cow<'_, str> {
    if s.contains(' ') {
        // فقط در صورت نیاز، رشته‌ی جدید ساخته می‌شود
        Cow::Owned(s.replace(' ', "_"))
    } else {
        Cow::Borrowed(s)
    }
}

fn main() {
    println!("{}", normalize("hello"));
    println!("{}", normalize("hello world"));
}
```

اشتباه رایج: نگه‌داشتن یک `borrow()` و هم‌زمان گرفتن `borrow_mut()` از همان `RefCell`؛ این کار در زمان اجرا باعث `panic` می‌شود.

درس ۵۰: مدیریت خطای حرفه‌ای

فرض این درس این است که `Result` و عملگر `?` را می‌شناسید.

تعریف خطای سفارشی

برای کتابخانه‌ها، خطا را با `enum` تعریف می‌کنیم و `Display` و `std::error::Error` را برایش پیاده‌سازی می‌کنیم. `From` تبدیل خودکار خطاها را برای `?` فراهم می‌کند.

```rust
use std::{error::Error, fmt, io};

#[derive(Debug)]
enum AppError {
    InvalidInput,
    Io(io::Error),
}

impl fmt::Display for AppError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::InvalidInput => write!(f, "ورودی نامعتبر"),
            Self::Io(e) => write!(f, "خطای فایل: {e}"),
        }
    }
}

impl Error for AppError {}

impl From<io::Error> for AppError {
    fn from(e: io::Error) -> Self {
        Self::Io(e)
    }
}

fn read_file() -> Result<String, AppError> {
    // خطای io::Error را با From به AppError تبدیل می‌کند ؟
    Ok(std::fs::read_to_string("input.txt")?)
```

```
}

fn main() {
    if let Err(e) = read_file() {
        eprintln!("{e}");
    }
    let _ = AppError::InvalidInput;
}
```

<Box<dyn Error

در برنامه‌های کوچک می‌توان انواع مختلف خطا را با `Box<dyn Error>` برگرداند:

```rust
use std::error::Error;

fn run() -> Result<(), Box<dyn Error>> {
    // خطای فایل به Box<dyn Error> تبدیل می‌شود
    let text = std::fs::read_to_string("input.txt")?;
    println!("{text}");
    Ok(())
}

fn main() {
    if let Err(e) = run() {
        eprintln!("{e}");
    }
}
```

thiserror و anyhow

- `thiserror`: برای library ها؛ خطاهای دقیق و مرتب تعریف می‌کند و کد تکراری `Display`، `Error` و `From` را خودکار تولید می‌کند.
- `anyhow`: برای application ها؛ انتشار خطا را ساده می‌کند و با `.context(...)` توضیح اضافه می‌کند.

هر دو crate خارجی‌اند و باید در `Cargo.toml` اضافه شوند:

```toml
[dependencies]
thiserror = "2"
anyhow = "1"
```

```rust
use anyhow::{Context, Result};
use thiserror::Error;

// خطای library با thiserror
#[derive(Debug, Error)]
enum AppError {
    #[error("ورودی نامعتبر")]
    InvalidInput,
    #[error("خطای فایل: {0}")]
    Io(#[from] std::io::Error),
}

fn load() -> Result<String, AppError> {
    Ok(std::fs::read_to_string("input.txt")?)
}

// خطای application با anyhow
fn main() -> Result<()> {
    let text = load().context("بارگذاری input.txt شکست خورد")?;
    println!("{text}");
    Ok(())
}
```

اشتباه رایج: استفاده از یک خطای عمومی برای همه‌ی خطاهای یک library؛ خطاهای مشخص، امکان مدیریت دقیق‌تر را به مصرف‌کننده می‌دهند.

درس ۵۱: تست و مستندسازی

Unit test و بررسی نتیجه

تابع دارای `#[test]` هنگام اجرای `cargo test` اجرا می‌شود. ماژول `#[cfg(test)]` فقط هنگام تست کامپایل می‌شود. از `assert!` برای شرط، `assert_eq!` برای برابری و `assert_ne!` برای نابرابری استفاده می‌کنیم.

```rust
// src/lib.rs
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn adds_numbers() {
        // نتیجه باید برابر 5 باشد
        assert_eq!(add(2, 3), 5);
        assert!(add(1, 1) > 0);
        assert_ne!(add(2, 2), 5);
    }

    #[test]
    #[should_panic]
    fn expected_panic() {
        panic!("خطای مورد انتظار");
    }
}
```

Integration test

تست‌های یکپارچه در پوشه‌ی `tests/` در کنار `src/` قرار می‌گیرند و crate را مثل یک مصرف‌کننده‌ی خارجی آزمایش می‌کنند. فقط API عمومی در دسترس آن‌هاست. توجه کنید که integration test ها و doc test ها به یک crate کتابخانه‌ای (`src/lib.rs`) نیاز دارند. فرض کنید نام crate در `Cargo.toml` برابر `my_app` است:

```
// tests/integration.rs
use my_app::add;

#[test]
fn public_api_works() {
    // API عمومی را از بیرون آزمایش می‌کنیم
    assert_eq!(add(10, 5), 15);
}
```

مستندسازی و doc test

مثال‌های داخل کامنت‌های `///` که در بلوک کد قرار دارند، هنگام `cargo test` به‌عنوان تست اجرا می‌شوند:

```
/// دو عدد را جمع می‌کند.
///
/// # Examples
///
/// ```
/// assert_eq!(my_app::add(2, 3), 5);
/// ```
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

- `cargo test` : اجرای unit test ها، integration test ها و doc test ها.
- `cargo doc --open` : تولید و نمایش مستندات HTML.
- `#[should_panic]` : موفقیت تست را به وقوع `panic` مورد انتظار وابسته می‌کند.

> **اشتباه رایج:** قرار دادن integration test در `src/`؛ این تست‌ها باید در پوشه‌ی جداگانه‌ی `tests/` باشند.

درس ۵۲: Sized و DST

Sized و نوع‌های بدون اندازه‌ی معلوم

trait ای به نام `Sized` یعنی اندازه‌ی نوع در زمان کامپایل مشخص است؛ بیشتر نوع‌های معمولی مثل `i32` و `String`، `Sized` هستند.

اما `str`، `[T]` و `dyn Trait` نوع‌های DST (dynamically sized type) هستند؛ اندازه‌ی آن‌ها در زمان کامپایل معلوم نیست. بنابراین نمی‌توان آن‌ها را مستقیماً به‌عنوان متغیر محلی نگه داشت؛ باید پشت یک pointer مثل `&str`، `&[T]` یا `Box<dyn Trait>` قرار بگیرند.

این pointer ها fat pointer هستند: علاوه بر آدرس داده، اطلاعات اضافه‌ای مثل طول یا vtable را هم نگه می‌دارند.

```
fn print_text(s: &str) {
    // str& شامل آدرس و طول متن است
    println!("{s}, length={}", s.len());
}

fn main() {
    let text: Box<str> = String::from("Rust").into_boxed_str();
    print_text(&text);
}
```

Sized? و slice مالکانه

پارامترهای generic به‌طور پیش‌فرض باید `Sized` باشند. با `?Sized` این الزام برداشته می‌شود؛ در نتیجه `T` می‌تواند DST باشد، اما همچنان باید از طریق reference یا pointer استفاده شود.

```rust
use std::fmt::Display;

// T می‌تواند نوعی بدون اندازه‌ی ثابت (مثل str) باشد
fn show<T: Display + ?Sized>(value: &T) {
    println!("{value}");
}

fn main() {
    show("Rust"); // T = str
    show(&42);    // T = i32

    // آرایه‌ی با اندازه‌ی ثابتِ تعیین‌شده در زمان ساخت
    let data: Box<[i32]> = vec![1, 2, 3].into_boxed_slice();
    println!("{}", data.len());
}
```

اشتباه رایج: تصور اینکه `?Sized` یعنی نوع الزاماً DST است؛ این قید فقط اجازه می‌دهد `T` غیر `Sized` هم باشد.

بخش پنجم: همزمانی و Async

درس ۵۳: Thread ها

ساخت thread با spawn و join

تابع `thread::spawn` یک thread جدید اجرا می‌کند و یک `JoinHandle` برمی‌گرداند. با `join()` منتظر پایان آن می‌مانیم و نتیجه‌اش را می‌گیریم. اگر thread دچار `panic` شود، `join()` مقدار `Err` برمی‌گرداند.

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| {
        // این کد در thread جداگانه اجرا می‌شود
        2 + 3
    });

    let result = handle.join().unwrap();
    println!("{result}"); // 5
}
```

چرا move لازم است؟

thread ممکن است بعد از پایان عمر متغیرهای تابع اصلی هم‌چنان اجرا شود؛ بنابراین closure باید مالک داده‌های موردنیازش باشد. `move` مالکیت متغیرهای captureشده را به closure منتقل می‌کند.

```
use std::thread;

fn main() {
    let message = String::from("سلام");

    let handle = thread::spawn(move || {
        // مالکیت message به closure منتقل شده است
        println!("{message}");
    });

    handle.join().unwrap();
}
```

قرض‌گرفتن با scope و مدیریت panic

`thread::scope` اجازه می‌دهد thread ها داده‌های محلی را قرض بگیرند؛ این تابع تضمین می‌کند همه‌ی thread های داخل scope پیش از خروج از آن تمام شوند.

```rust
use std::thread;

fn main() {
    let numbers = [10, 20, 30];

    thread::scope(|s| {
        s.spawn(|| {
            // قرض‌گرفتن numbers بدون انتقال مالکیت
            println!("{}", numbers[0]);
        });

        let handle = s.spawn(|| {
            panic!("خطای thread");
        });

        // گزارش می‌شود Err به‌صورت thread در panic
        assert!(handle.join().is_err());
    });
}
```

اشتباه رایج: فرض اینکه `spawn` به‌تنهایی منتظر پایان thread می‌ماند؛ برای دریافت نتیجه و تشخیص `panic` از `join()` استفاده کنید.

درس ۵۴: ارتباط و اشتراک داده بین thread ها

فرض این درس این است که `thread::spawn` و `Rc` را می‌شناسید.

کانال mpsc

کانال برای ارسال پیام بین thread ها است. `mpsc` یعنی «چند فرستنده، یک گیرنده» (multiple producer, single consumer). با `send` پیام می‌فرستیم و با `recv` آن را دریافت می‌کنیم.

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        // ارسال پیام به thread اصلی
        tx.send(42).unwrap();
    });

    println!("{}", rx.recv().unwrap());
}
```

اشتراک داده با Arc<Mutex<T>>

`Mutex<T>` اجازه می‌دهد در هر لحظه فقط یک thread به داده دسترسی داشته باشد. `Arc<T>` مالکیت مشترک و thread-safe فراهم می‌کند؛ ترکیب این دو برای داده‌ی قابل‌تغییر مشترک رایج است.

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..3 {
        let c = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            // قفل، دسترسی همزمان را کنترل می‌کند
            *c.lock().unwrap() += 1;
        }));
    }

    for h in handles {
        h.join().unwrap();
    }
    println!("{}", *counter.lock().unwrap()); // 3
}
```

atomic، RwLock و Send/Sync

- `RwLock<T>` چند خواننده یا فقط یک نویسنده را هم‌زمان مجاز می‌کند.
- atomic هایی مانند `AtomicUsize` عملیات ساده‌ای مثل افزایش را بدون `Mutex` انجام می‌دهند.
- `Send` یعنی مقدار را می‌توان به thread دیگر منتقل کرد و `Sync` یعنی می‌توان reference به آن را بین thread ها به اشتراک گذاشت. کامپایلر با این trait ها بسیاری از اشتراک‌های ناامن را رد می‌کند.

```rust
use std::sync::atomic::{AtomicUsize, Ordering};
use std::sync::{Arc, RwLock};
use std::thread;

fn main() {
    // atomic
    let n = Arc::new(AtomicUsize::new(0));
    let c = Arc::clone(&n);
    thread::spawn(move || {
        c.fetch_add(1, Ordering::Relaxed);
    })
    .join()
    .unwrap();
    println!("{}", n.load(Ordering::Relaxed)); // 1

    // RwLock: چند خواننده یا یک نویسنده
    let data = RwLock::new(5);
    {
        let r1 = data.read().unwrap();
        let r2 = data.read().unwrap();
        println!("{} {}", *r1, *r2);
    } // خواننده‌ها آزاد شدند
    *data.write().unwrap() += 1;
    println!("{}", *data.read().unwrap()); // 6
}
```

کامپایلر استفاده از `Rc` را در thread رد می‌کند، چون `Rc` نه `Send` است و نه `Sync`:

```
use std::rc::Rc;
use std::thread;

fn main() {
    let data = Rc::new(5);
    // Rc<i32>` cannot be sent between threads safely` :خطای کامپایل
    // thread::spawn(move || println!("{data}"));
    let _ = (data, thread::current());
}
```

اشتباه رایج: `Arc` به‌تنهایی تغییر امن داده‌ی مشترک را تضمین نمی‌کند؛ برای داده‌ی قابل‌تغییر از ابزار همگام‌سازی مناسب (مثل `Mutex`) استفاده کنید.

درس ۵۵: Async/await (پایه)

async fn و await.

تابع `async fn` به‌جای اجرای فوری، یک `Future` برمی‌گرداند. Future تنبل است؛ تا وقتی با `.await` اجرا نشود، بدنه‌اش پیش نمی‌رود. خود Rust runtime ندارد؛ برای اجرای کد async معمولاً از کتابخانه‌ای مانند Tokio استفاده می‌کنیم.

در `Cargo.toml`:

```
[dependencies]
tokio = { version = "1", features = ["macros", "rt-multi-thread", "time"] }
```

```
use tokio::time::{sleep, Duration};

async fn work() -> i32 {
    // هنگام await، اجرا به runtime برمی‌گردد و بعد از 100ms ادامه می‌یابد
    sleep(Duration::from_millis(100)).await;
    42
}

#[tokio::main]
async fn main() {
    let result = work().await;
    println!("{result}");
}
```

async block و async move

عبارت `async { ... }` یک Future می‌سازد. با `async move`، متغیرهای captureشده به مالکیت آن منتقل می‌شوند.

```rust
#[tokio::main]
async fn main() {
    let name = String::from("Rust");

    let task = async move {
        // name به async block منتقل شده است
        name.len()
    };

    println!("{}", task.await);
}
```

Future و async closure

تریت `Future` پایه‌ی همه‌ی عملیات async است. در Rust پایدار (از نسخه‌ی ۱٫۸۵)، async closure با `async || ...` نوشته می‌شود.

```rust
#[tokio::main]
async fn main() {
    let add = async |x: i32| x + 1;

    // فراخوانی closure یک Future می‌دهد
    println!("{}", add(4).await);
}
```

اشتباه رایج: تصور اینکه `async` به‌تنهایی کد را در thread جداگانه اجرا می‌کند؛ اجرای Future به runtime یا executor نیاز دارد.

درس ۵۶: Async (پیشرفته)

فرض این درس این است که `async` / `await` پایه را می‌شناسید.

Pin و Unpin

بعضی Future ها ممکن است به داده‌های داخل خودشان reference داشته باشند (self-referential). جابه‌جایی چنین مقداری در حافظه می‌تواند آن reference ها را نامعتبر کند. `Pin` مانع جابه‌جایی مقدار می‌شود و `Unpin` یعنی نوع این محدودیت را ندارد و جابه‌جایی‌اش ایمن است. در کد async روزمره معمولاً مستقیم با `Pin` کار ندارید؛ فقط گاهی هنگام برگرداندن future ها به‌صورت trait object با آن روبه‌رو می‌شوید:

```
use std::future::Future;
use std::pin::Pin;

// future با نوع پنهان که روی heap قرار دارد و جابه‌جا نمی‌شود
fn make() -> Pin<Box<dyn Future<Output = i32>>> {
    Box::pin(async { 42 })
}

#[tokio::main]
async fn main() {
    println!("{}", make().await);
}
```

اجرای هم‌زمان Future ها

در `Cargo.toml`:

```
[dependencies]
tokio = { version = "1", features = ["macros", "rt-multi-thread", "time"] }
```

```
use tokio::time::{sleep, Duration};

async fn job(ms: u64) -> u64 {
    sleep(Duration::from_millis(ms)).await;
    ms
}

#[tokio::main]
async fn main() {
    // هر دو future هم‌زمان پیش می‌روند؛ زمان کل حدود 200ms است، نه 300ms
    let (a, b) = tokio::join!(job(100), job(200));
    println!("{a} {b}");

    // منتظر اولین شاخه‌ی آماده می‌ماند و بقیه را لغو می‌کند :!select
    tokio::select! {
        _ = sleep(Duration::from_millis(50)) => println!("اولی تمام شد"),
        _ = sleep(Duration::from_millis(500)) => println!("دومی تمام شد"),
    }
}
```

tokio::spawn و Send

```
use tokio::time::{sleep, Duration};

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async move {
        sleep(Duration::from_millis(50)).await;
        42
    });

    println!("{}", handle.await.unwrap());
}
```

`tokio::spawn` معمولاً به `Future + Send + 'static` نیاز دارد، چون task ممکن است بین thread های runtime جابه‌جا شود. لغو (cancellation) یک Future معمولاً با رهاکردن (drop) آن انجام می‌شود و عملیات نیمه‌تمام در نقطه‌ی `.await` متوقف می‌شود.

انتخاب بین async و thread: `async` برای تعداد زیاد عملیات انتظارمحور مثل شبکه مناسب است. `thread` برای اجرای موازی کارهای CPUمحور یا کد همگام مسدودکننده مناسب است.

> **اشتباه رایج:** `async` به‌معنای اجرای موازی نیست؛ یک task سنگین CPUمحور می‌تواند thread ی runtime را مسدود کند. برای چنین کارهایی از `tokio::task::spawn_blocking` استفاده کنید.

بخش ششم: Attribute و ماکرو

درس ۵۷: Attribute ها (عمیق) و کامپایل شرطی

فرض این درس این است که شکل `#[...]` و `#![...]` و `#[derive]` را می‌شناسید.

cfg، cfg_attr و cfg!

- `#[cfg(...)]` آیتم را فقط در صورت برقرار بودن شرط وارد کامپایل می‌کند.
- `#[cfg_attr(condition, attribute)]` در صورت برقراری شرط، attribute دیگری را اعمال می‌کند.
- `cfg!(...)` مقدار `true` یا `false` تولید می‌کند؛ کد هر دو شاخه همچنان کامپایل می‌شود.

```rust
// این تابع فقط در Linux کامپایل می‌شود
#[cfg(target_os = "linux")]
fn platform() {
    println!("Linux");
}

// فقط در Windows این attribute اعمال می‌شود
#[cfg_attr(target_os = "windows", allow(dead_code))]
fn helper() {}

fn main() {
    #[cfg(target_os = "linux")]
    platform();

    // ترکیب شرط‌ها با all، any و not
    if cfg!(all(target_os = "linux", not(target_os = "android"))) {
        println!("Linux (نه Android)");
    }

    // این feature باید در Cargo.toml تعریف شده باشد
    #[cfg(feature = "fast")]
    println!("قابلیت fast فعال است");
}
```

شرط‌های `all`، `any` و `not` به‌ترتیب AND، OR و NOT هستند؛ `true` و `false` نیز شرط‌های ثابت‌اند. اگر نام یک feature را در فایل `Cargo.toml` تعریف نکرده باشید، کامپایلر هشدار `unexpected_cfgs` می‌دهد.

attribute های مهم

```
#[must_use] // نادیده‌گرفتن نتیجه هشدار می‌دهد
#[inline]   // پیشنهاد inline کردن به کامپایلر
fn calculate() -> i32 {
    42
}

#[deprecated(since = "1.0.0", note = "از new_api استفاده کنید")]
fn old_api() {}

// کد مصرف‌کننده‌ی crate باید حالت‌های ناشناخته را هم در match پوشش دهد
#[non_exhaustive]
pub enum Color {
    Red,
    Green,
    Blue,
}

fn main() {
    let _ = calculate();
}
```

نمایش حافظه و attribute ناامن

```
#[repr(C)] // چیدمان سازگار با قواعد C
struct Point {
    x: i32,
    y: i32,
}

// در edition 2024، این attribute باید داخل unsafe(...) نوشته شود
#[unsafe(no_mangle)]
pub extern "C" fn plugin_entry() {}
```

`#[repr(...)]` چیدمان یا نمایش نوع را کنترل می‌کند. `no_mangle` نام نماد (symbol) تابع را تغییر نمی‌دهد؛ چون ممکن است باعث تداخل نام شود، در edition 2024 به‌صورت `unsafe` علامت‌گذاری می‌شود و مسئولیت جلوگیری از تداخل با شماست.

> **اشتباه رایج:** تصور اینکه `cfg!` کد را حذف می‌کند؛ `cfg!` فقط یک مقدار `bool` می‌دهد و هر دو شاخه کامپایل می‌شوند. برای حذف آیتم از کامپایل، از `#[cfg(...)]` استفاده کنید.

درس ۵۸: نوشتن ماکرو با macro_rules!

فرض این درس این است که استفاده از ماکروهایی مثل `println!` و `vec!` را می‌شناسید.

ساختار macro_rules! و fragment ها

ماکرو در زمان کامپایل کد تولید می‌کند. در matcher، متغیرهایی مثل `$x:expr` بخش‌هایی از ورودی را می‌گیرند و در خروجی جایگزین می‌شوند.

fragment specifier های پرکاربرد: `expr` (عبارت)، `ident` (شناسه)، `ty` (نوع)، `pat` (الگو)، `stmt` (دستور)، `block` (بلوک)، `item` (آیتم)، `literal` (مقدار ثابت)، `path` (مسیر) و `tt` (یک token tree).

```
// expr ورودی را به‌عنوان یک عبارت کامل دریافت می‌کند
macro_rules! twice {
    ($x:expr) => {
        $x * 2
    };
}

fn main() {
    // ورودی به‌صورت گروه‌بندی‌شده جایگزین می‌شود: 2 * (1 + 5)
    println!("{}", twice!(5 + 1)); // 12
}
```

تکرار و جداکننده

`$(...)*` صفر یا چند بار، `$(...)+` یک یا چند بار و `$(...)?` صفر یا یک بار تکرار را مشخص می‌کند. جداکننده (مثل `,`) قبل از علامت تکرار نوشته می‌شود.

```
// چند عبارت جداشده با ویرگول دریافت می‌کند و جمعشان را برمی‌گرداند
macro_rules! sum {
    ($($x:expr),* $(,)?) => {
        0 $(+ $x)*
    };
}

fn main() {
    println!("{}", sum!(1, 2, 3)); // 6
    println!("{}", sum!());        // 0
}
```

ماکروی بازگشتی و hygiene

ماکرو می‌تواند خودش را فراخوانی کند؛ مثلاً برای شمارش ورودی‌ها:

```
macro_rules! count {
    () => { 0usize };
    ($head:tt $($tail:tt)*) => { 1usize + count!($($tail)*) };
}

fn main() {
    println!("{}", count!(a b c)); // 3
}
```

Hygiene یعنی نام‌های محلی داخل ماکرو با نام‌های فراخواننده تداخل نمی‌کنند:

```
macro_rules! make_value {
    () => {{
        let x = 42; // x داخلی ماکرو
        x
    }};
}

fn main() {
    let x = 1;
    let y = make_value!();
    println!("{x} {y}"); // بیرونی دست‌نخورده می‌ماند x ؛42 1
}
```

اشتباه رایج: استفاده از `*` به‌جای `+` وقتی حداقل یک ورودی لازم است؛ `*` ورودی خالی را هم می‌پذیرد.

درس ۵۹: Procedural macro

فرض این درس این است که `macro_rules!` و trait ها را می‌شناسید.

سه نوع procedural macro

برخلاف `macro_rules!`، در procedural macro ورودی به‌صورت یک `TokenStream` دریافت می‌شود و خروجی کد Rust است که با کد Rust نوشته شده است.

- Derive: با `#[derive(MyTrait)]` پیاده‌سازی یک trait را تولید می‌کند.

- Attribute-like: با `#[my_attribute]` روی یک آیتم اعمال می‌شود.
- Function-like: مانند `my_macro!(...)` فراخوانی می‌شود.

ساختار crate

این ماکروها باید در crate جداگانه‌ای تعریف شوند که در `Cargo.toml` آن `proc-macro = true` باشد. کتابخانه‌ی `syn` برای تجزیه‌ی ورودی و `quote` برای تولید کد به کار می‌رود.

```
# my_derive/Cargo.toml
[package]
name = "my_derive"
version = "0.1.0"
edition = "2024"

[lib]
proc-macro = true

[dependencies]
syn = { version = "2", features = ["full"] }
quote = "1"
```

مثال derive

در `my_derive/src/lib.rs`:

```
use proc_macro::TokenStream;
use quote::quote;
use syn::{parse_macro_input, DeriveInput};

#[proc_macro_derive(Hello)]
pub fn derive_hello(input: TokenStream) -> TokenStream {
    let input = parse_macro_input!(input as DeriveInput);
    let name = input.ident;

    // پیاده‌سازی trait برای نوع ورودی تولید می‌شود
    quote! {
        impl HelloTrait for #name {
            fn hello() {
                println!("Hello from {}", stringify!(#name));
            }
        }
    }
    .into()
}
```

در crate مصرف‌کننده، ابتدا وابستگی را در `Cargo.toml` اضافه کنید:

```
[dependencies]
my_derive = { path = "../my_derive" }
```

سپس:

```
use my_derive::Hello;

trait HelloTrait {
    fn hello();
}

#[derive(Hello)]
struct User;

fn main() {
    User::hello(); // Hello from User
}
```

اشتباه رایج: تعریف `proc-macro = true` در crate برنامه‌ی اصلی؛ procedural macro باید در crate مخصوص خودش تعریف شود.

بخش هفتم: سیستم نوع پیشرفته و سطح پایین

درس ۶۰: Trait های پیشرفته

فرض این درس این است که associated type، generic، trait و trait object را می‌شناسید.

قید روی associated type

با `Trait<Item: Send>` می‌توان علاوه بر الزام پیاده‌سازی trait، روی نوع associated آن هم قید گذاشت.

```rust
use std::fmt::Debug;

trait Source {
    type Item;
    fn next(&mut self) -> Option<Self::Item>;
}

// نوع Item باید هم Debug و هم Send باشد
fn read<S: Source<Item: Debug + Send>>(source: &mut S) {
    println!("{:?}", source.next());
}

struct Numbers;

impl Source for Numbers {
    type Item = i32;

    fn next(&mut self) -> Option<i32> {
        Some(42)
    }
}

fn main() {
    read(&mut Numbers);
}
```

GAT (generic associated type) اجازه می‌دهد associated type خودش پارامتر generic داشته باشد؛ مثلاً نوعی که به lifetime یک borrow وابسته است.

```
trait Container {
    // نوع وابسته به lifetime
    type View<'a>
    where
        Self: 'a;

    fn view(&self) -> Self::View<'_>;
}

impl Container for String {
    type View<'a> = &'a str;

    fn view(&self) -> Self::View<'_> {
        self.as_str()
    }
}

fn main() {
    let text = String::from("Rust");
    println!("{}", text.view());
}
```

Marker trait نشان‌دهنده‌ی یک ویژگی است، بدون اینکه متدی داشته باشد؛ مثل `Copy`، `Send` و `Sync`. Auto trait ها مانند `Send` و `Sync` در بسیاری موارد به‌طور خودکار از اجزای یک نوع استنباط می‌شوند.

Coherence و orphan rule

نمی‌توان هر trait را برای هر نوعی پیاده‌سازی کرد؛ معمولاً باید trait یا نوع متعلق به crate خودتان باشد. این قانون از پیاده‌سازی‌های متعارض جلوگیری می‌کند.

`impl !Trait for Type` برای رد صریح یک auto trait است. این قابلیت (negative impl) هنوز پایدار نیست و در کد معمولی فقط در کتابخانه‌ی استاندارد دیده می‌شود.

> **اشتباه رایج:** تصور اینکه می‌توان trait خارجی را برای نوع خارجی پیاده‌سازی کرد؛ این کار معمولاً قانون orphan را نقض می‌کند.

درس ۶۱: نوع ! و تابع diverging

نوع ! و تابعی که برنمی‌گردد

نوع `!` (never type) نشان می‌دهد که یک expression هرگز مقداری تولید نمی‌کند؛ مثلاً چون برنامه متوقف می‌شود یا اجرای فعلی هیچ‌گاه به ادامه نمی‌رسد.

```rust
fn stop() -> ! {
    // این تابع هرگز برنمی‌گردد
    panic!("برنامه متوقف شد");
}

fn main() {
    let args = std::env::args().count();
    if args > 100 {
        stop();
    }
    println!("ادامه");
}
```

عبارت‌های `panic!()`، `loop {}` بی‌نهایت، `continue`، `break` و `return` همگی از نوع `!` هستند؛ چون اجرای مسیر فعلی را ادامه نمی‌دهند.

سازگاری با هر نوع در match

چون `!` هرگز مقداری تولید نمی‌کند، به هر نوعی تبدیل می‌شود و می‌تواند در شاخه‌ای بیاید که شاخه‌های دیگر نوع مشخصی دارند.

```
fn value(ok: bool) -> i32 {
    match ok {
        true => 42,
        false => panic!("خطا"), // شاخه‌ی ! با i32 سازگار است
    }
}

fn main() {
    println!("{}", value(true));
}
```

سه کاربرد متفاوت علامت !

```
fn main() {
    let active = true;

    // ۱. عملگر NOT برای bool
    println!("{}", !active); // false

    // ۲. علامت فراخوانی ماکرو
    println!("Hello");

    // ۳. نوع !: شاخه‌ی return از نوع ! است و با i32 سازگار می‌شود
    let number: i32 = if active { 10 } else { return };
    println!("{number}");
}
```

اشتباه رایج: `!` در `bool` عملگر NOT است، اما `!` به‌عنوان نوع یعنی آن expression هرگز مقداری برنمی‌گرداند.

درس ۶۲: Const evaluation

فرض این درس این است که generic ها و `const` / `static` پایه را می‌شناسید.

const fn و const { ... }

تابع `const fn` می‌تواند در زمان کامپایل اجرا شود، اگر در یک زمینه‌ی ثابت (مثل مقدار یک `const`) استفاده شود و عملیاتش مجاز باشد. بلوک `const { ... }` مقدار یک عبارت را در زمان کامپایل محاسبه می‌کند.

```
const fn square(n: usize) -> usize {
    n * n
}

const SIZE: usize = square(4);

fn main() {
    // طول آرایه در زمان کامپایل محاسبه می‌شود
    let values = [0; SIZE];
    println!("{}", values.len()); // 16

    let limit = const { 10 * 2 };
    println!("{limit}"); // 20
}
```

const generics

با `const N: usize` می‌توان یک مقدار ثابت (مثل اندازه) را پارامتر نوع قرار داد؛ بنابراین هر مقدار، نوعی متفاوت می‌سازد.

```
struct Buf<const N: usize> {
    data: [u8; N],
}

fn size_of_array<const N: usize>(_arr: [i32; N]) -> usize {
    N
}

fn main() {
    let a = Buf::<4> { data: [0; 4] };
    let b = Buf::<8> { data: [1; 8] };

    println!("{} {}", a.data.len(), b.data.len());
    println!("{}", size_of_array([1, 2, 3])); // 3
}
```

مقدار ثابت استنباط‌شده و محدودیت‌ها

در برخی موقعیت‌ها می‌توان مقدار const generic را با `_` به کامپایلر سپرد:

```
struct Array<const N: usize>([i32; N]);

fn main() {
    // از آرایه‌ی ورودی استنباط می‌شود N
    let a: Array<_> = Array([1, 2, 3]);
    println!("{}", a.0.len());
}
```

طول آرایه می‌تواند از یک const expression به دست آید. اما `const fn` نمی‌تواند هر عملیات زمان اجرا را انجام دهد؛ مثلاً عملیات I/O یا فراخوانی تابعی که const نیست.

> **اشتباه رایج:** `const fn` به این معنا نیست که هر فراخوانی تابع حتماً در زمان کامپایل اجرا می‌شود؛ در زمینه‌ی غیرثابت، همان تابع در زمان اجرا هم فراخوانی می‌شود.

درس ۶۳: الگوهای نوع

فرض این درس این است که generic ها و trait ها را می‌شناسید.

type alias و الگوی newtype

`type Name = Type;` فقط یک نام مستعار می‌سازد؛ دو نام همچنان یک نوع‌اند. اما newtype، یک struct تک‌فیلدی است که نوعی واقعاً جدید ایجاد می‌کند.

```rust
use std::marker::PhantomData;

type UserId = u64; // فقط نام مستعار؛ همان u64 است

struct Meters(f64); // newtype؛ با f64 ساده سازگار نیست

// PhantomData برای نمایش ارتباط نوعی بدون نگهداری داده‌ی واقعی
struct Id<T> {
    value: u64,
    _marker: PhantomData<T>,
}

fn main() {
    let id: UserId = 42;
    let distance = Meters(12.5);
    let typed_id: Id<String> = Id {
        value: id,
        _marker: PhantomData,
    };
    println!("{} {} {}", id, distance.0, typed_id.value);
}
```

zero-sized type و typestate

نوع بدون اندازه‌ی واقعی (ZST) مثل `()` یا `PhantomData<T>` داده‌ی معمولی ذخیره نمی‌کند؛ برای علامت‌گذاری در سطح نوع مفید است.

در الگوی typestate، نوع Rust وضعیت شیء را مشخص می‌کند؛ در نتیجه عملیات نامعتبر در زمان کامپایل رد می‌شود.

```rust
use std::marker::PhantomData;

struct Closed;
struct Open;

struct Connection<State> {
    _state: PhantomData<State>,
}

impl Connection<Closed> {
    fn open(self) -> Connection<Open> {
        Connection { _state: PhantomData }
    }
}

impl Connection<Open> {
    fn send(&self) {
        println!("Sent");
    }
}

fn main() {
    let connection = Connection::<Closed> {
        _state: PhantomData,
    };
    // وجود دارد <Connection<Open فقط برای send :خطای کامپایل // ;()connection.send
```

```
    connection.open().send();
}
```

union

`union` نوعی است که فیلدهایش حافظه‌ی مشترک دارند. خواندن یک فیلد آن نیازمند `unsafe` است؛ بنابراین در کد معمولی کمتر استفاده می‌شود و در درس ۶۵ بررسی می‌شود.

> **اشتباه رایج:** تصور اینکه `type UserId = u64` نوعی جدا از `u64` می‌سازد؛ برای جداسازی واقعی از `struct UserId(u64)` استفاده کنید.

درس ۶۴: Memory layout

فرض این درس این است که enum، struct، ownership و `Sized` را می‌شناسید.

اندازه، هم‌ترازی و repr

`size_of::<T>()` اندازه‌ی نوع و `align_of::<T>()` هم‌ترازی (alignment) موردنیاز آن را برحسب بایت می‌دهد.

- `#[repr(Rust)]`: حالت پیش‌فرض؛ چیدمان تضمین‌شده نیست.
- `#[repr(C)]`: چیدمان سازگار با قواعد C.
- `#[repr(transparent)]`: حفظ چیدمان نوعِ تک‌فیلدی‌ای که فقط یک فیلد غیر ZST دارد.
- `#[repr(packed)]`: حذف padding؛ دسترسی به فیلدهای ناهم‌تراز خطرناک است.
- `#[repr(align(N))]`: حداقل هم‌ترازی را افزایش می‌دهد.

```rust
use std::mem::{align_of, size_of};

#[repr(C)]
struct Data {
    a: u8,
    b: u32,
}

#[repr(C, packed)]
struct Packed {
    a: u8,
    b: u32,
}

#[repr(align(16))]
struct Aligned(u8);

fn main() {
    // 5 نه ،شود 8 اندازه میشود باعث padding
    println!("{} {}", size_of::<Data>(), align_of::<Data>());        // 8 4
    println!("{}", size_of::<Packed>());                              // 5
    println!("{} {}", size_of::<Aligned>(), align_of::<Aligned>());  // 16 16
}
```

گرفتن reference به فیلد ناهم‌ترازِ یک struct با `packed` خطای کامپایل است؛ مقدار آن را باید کپی کرد.

Niche optimization و ترتیب drop

کامپایلر می‌تواند از الگوی بیتیِ نامعتبر یک نوع برای نمایش حالت دیگری استفاده کند. برای نمونه، `Option<&i32>` معمولاً هم‌اندازه‌ی `&i32` است، چون یک reference معتبر هرگز null نیست. این بهینه‌سازی برای همه‌ی نوع‌ها تضمین‌شده نیست.

```
use std::mem::size_of;

fn main() {
    // معمولاً هر دو اندازه برابرند
    println!("{} {}", size_of::<&i32>(), size_of::<Option<&i32>>());
}
```

فیلدهای struct به ترتیب تعریف drop می‌شوند، اما متغیرهای محلی به ترتیب معکوس تعریف:

```rust
use std::mem::ManuallyDrop;

struct Noisy(&'static str);

impl Drop for Noisy {
    fn drop(&mut self) {
        println!("drop {}", self.0);
    }
}

struct Pair {
    a: Noisy,
    b: Noisy,
}

fn main() {
    let _p = Pair { a: Noisy("a"), b: Noisy("b") };
    let _x = Noisy("x");
    // ManuallyDrop مانع drop خودکار می‌شود (مقدار هرگز آزاد نمی‌شود)
    let _keep = ManuallyDrop::new(Noisy("manual"));
}
// خروجی: drop x، drop a، drop b
```

جابه‌جایی مقدارها

```rust
use std::mem;

fn main() {
    let mut a = String::from("A");
    let mut b = String::from("B");

    mem::swap(&mut a, &mut b);                 // "a = "B"، b = "A
    let old = mem::replace(&mut a, String::from("C")); // "old = "B"، a = "C
    let taken = mem::take(&mut b);             // (مقدار پیش‌فرض) "" = taken = "A"، b

    println!("{old} {taken} {a}"); // B A C
}
```

استفاده‌ی نادرست از `ManuallyDrop` می‌تواند منابع را نشت دهد یا باعث رفتار تعریف‌نشده شود.

> **اشتباه رایج:** گرفتن reference از فیلد ناهم‌ترازِ struct با `#[repr(packed)]`؛ این کار خطای کامپایل می‌دهد و باید مقدار را کپی کرد.

درس ۶۵: unsafe

فرض این درس این است که ownership، reference و trait ها را می‌شناسید.

پنج کاری که فقط در unsafe مجاز است

در Rust، `unsafe` مسئولیت رعایت برخی قواعد ایمنی را از کامپایلر به برنامه‌نویس منتقل می‌کند. پنج مورد اصلی:

1. dereference کردن raw pointer.
2. فراخوانی `unsafe fn`.
3. دسترسی به فیلدهای `union`.
4. دسترسی یا تغییر `static mut`.
5. پیاده‌سازی `unsafe trait`.

`unsafe` تضمین نمی‌کند کد درست است؛ فقط اجازه‌ی انجام این عملیات را می‌دهد و مسئولیت درستی با شماست.

Raw pointer و unsafe block

نوع `*const T` اشاره‌گر فقط‌خواندنی و `*mut T` اشاره‌گر قابل‌تغییر است. برخلاف reference ها، این اشاره‌گرها لزوماً معتبر یا غیر null نیستند. عبارت‌های `&raw const x` و `&raw mut x` یک raw pointer می‌سازند، بدون اینکه reference معمولی ایجاد کنند.

```
fn main() {
    let mut value = 10;

    // ساختن raw pointer ایمن است؛ فقط dereference کردن آن ناامن است
    let ptr: *mut i32 = &raw mut value;

    unsafe {
        // معتبر بودن pointer بر عهده‌ی ماست
        *ptr += 5;
    }

    println!("{value}"); // 15
}
```

unsafe fn و wrapper امن

اصل مهم: یک wrapper امن بنویسید و بلوک `unsafe` را تا حد ممکن کوچک نگه دارید.

```
// فراخواننده باید تضمین کند که ptr معتبر است
unsafe fn read_ptr(ptr: *const i32) -> i32 {
    unsafe { *ptr }
}

// wrapper امن: ورودی reference است و حتماً معتبر است
fn read_value(value: &i32) -> i32 {
    unsafe { read_ptr(value as *const i32) }
}

fn main() {
    let n = 42;
    println!("{}", read_value(&n));
}
```

در edition 2024، عملیات ناامن داخل `unsafe fn` نیز باید در یک بلوک `unsafe` مشخص شوند (همان‌طور که در مثال بالا دیده می‌شود).

static mut، union و unsafe trait

```
// فیلدها حافظه‌ی مشترک دارند :union
union IntOrFloat {
    i: u32,
    f: f32,
}

// قراردادی که کامپایلر نمی‌تواند صحتش را بررسی کند :unsafe trait
unsafe trait Zeroable {}
unsafe impl Zeroable for u32 {}

static mut COUNTER: u32 = 0;

fn main() {
    let u = IntOrFloat { f: 1.0 };
    let bits = unsafe { u.i }; // خواندن فیلد union ناامن است
    println!("{bits:#x}");     // 0x3f800000

    unsafe {
        COUNTER += 1; // دسترسی مستقیم به static mut
        // در edition 2024 گرفتن reference معمولی (&COUNTER) خطای کامپایل است؛
        // به‌جای آن از raw pointer استفاده کنید:
        let p = &raw const COUNTER;
        println!("{}", *p);
    }
}
```

تا حد امکان از `static mut` پرهیز کنید و به‌جای آن از ابزارهایی مثل `Mutex` یا atomic استفاده کنید.

اشتباه رایج: کوچک‌بودن بلوک `unsafe` به‌تنهایی ایمنی را تضمین نمی‌کند؛ باید اعتبار pointer، طول عمر و هم‌زمانی را هم بررسی کنید.

درس ۶۶: FFI (ارتباط با C)

فرض این درس این است که `unsafe`، raw pointer و `repr` را می‌شناسید.

فراخوانی تابع C از Rust

در edition 2024، بلوک خارجی باید `unsafe extern` باشد، چون کامپایلر نمی‌تواند درستی امضای تابع خارجی را بررسی کند. هر آیتم داخل آن را می‌توان `safe` یا `unsafe` علامت زد. تابع‌های variadic با `...` نیز در این بلوک اعلام می‌شوند.

```rust
use std::ffi::{c_char, c_int, CString};

unsafe extern "C" {
    // قرارداد این تابع ساده است و فراخوانی آن امن است
    safe fn abs(n: c_int) -> c_int;
    // باشد null-terminated و معتبر باید pointer
    unsafe fn strlen(s: *const c_char) -> usize;
    // variadic تابع
    unsafe fn printf(format: *const c_char, ...) -> c_int;
}

fn main() {
    println!("{}", abs(-7));

    // C رشته‌ی مالکانه‌ی سازگار با :CString
    let owned = CString::new("Rust").unwrap();
    let len = unsafe { strlen(owned.as_ptr()) };
    println!("{len}");

    // است C string literal یک "..."c
    unsafe { printf(c"value: %d\n".as_ptr(), 42) };
}
```

صادرکردن تابع و رشته‌های C

`extern "C" fn` تابعی با calling convention زبان C تعریف می‌کند. در edition 2024، `no_mangle` باید به‌شکل `#[unsafe(no_mangle)]` نوشته شود. `CString` رشته‌ی مالکانه‌ی سازگار با C و `CStr` نمای borrowed آن است.

```rust
use std::ffi::CStr;

// این تابع از طریق نماد C با نام add قابل دسترسی است
#[unsafe(no_mangle)]
pub extern "C" fn add(a: i32, b: i32) -> i32 {
    a + b
}

fn main() {
    let s: &CStr = c"hello";
    println!("{:?} {}", s, add(1, 2));
}
```

برای اینکه کد Rust به‌صورت کتابخانه‌ی مشترک برای C در دسترس باشد، در `Cargo.toml` باید `crate-type = ["cdylib"]` تعریف شود.

ابزارها و قابلیت‌های دیگر

- `#[repr(C)]` : چیدمان struct را برای سازگاری با C مشخص می‌کند.
- تعریف تابع variadic در Rust پایدار ممکن نیست؛ فقط می‌توان توابع variadic خارجی را (مثل `printf`) فراخوانی کرد.
- `bindgen` : از headerهای C/C++ کد binding برای Rust تولید می‌کند.
- `cbindgen` : از API های Rust، headerهای C/C++ تولید می‌کند.

> **اشتباه رایج:** رشته‌ی معمولی Rust (`String` یا `&str`) را مستقیماً به C ندهید؛ این رشته‌ها NUL-terminated نیستند. از `CString` و `CStr` استفاده کنید و مراقب طول عمر pointer باشید.

درس ۶۷: Inline assembly

فرض این درس این است که `unsafe` را می‌شناسید. مثال‌ها برای معماری x86_64 هستند.

ماکروی asm! و operand ها

ماکروی `asm!` دستور اسمبلی را داخل تابع Rust اجرا می‌کند و به `unsafe` نیاز دارد. نوع‌های اصلی operand:

- `in(reg)` : مقدار ورودی در register.
- `out(reg)` : خروجی در register.
- `inout(reg)` : ورودی و خروجی مشترک.
- `const` : مقدار ثابت در زمان کامپایل.
- `sym` : ارجاع به نماد یک تابع یا متغیر static.
- clobber: registerهایی که اسمبلی تغییر می‌دهد باید اعلام شوند؛ با `out("rax") _` یا `clobber_abi("C")` (نحو مستقلی به نام `clobber` وجود ندارد).

```
use std::arch::asm;

fn main() {
    let a: u64 = 7;
    let b: u64 = 5;
    let result: u64;

    unsafe {
        // جمع دو مقدار در registerها
        asm!(
            "add {0}, {1}",
            inout(reg) a => result,
            in(reg) b,
        );
    }

    println!("{result}"); // 12
}
```

خروجی، const و option ها

```
use std::arch::asm;

fn main() {
    let result: u64;

    unsafe {
        // یک مقدار ثابت در register خروجی قرار می‌گیرد
        asm!(
            "mov {res}, {c}",
            res = out(reg) result,
            c = const 42,
            options(nomem, nostack, pure),
        );
    }

    println!("{result}"); // 42
}
```

- `nomem` : اسمبلی به حافظه نمی‌نویسد و از آن نمی‌خواند.
- `nostack` : از stack استفاده نمی‌کند.
- `pure` : اثر جانبی ندارد و خروجی فقط به ورودی‌ها وابسته است (باید حداقل یک خروجی وجود داشته باشد).
- `raw` : در template، علامت‌های `{` و `}` به‌عنوان فرمت‌بندی تفسیر نمی‌شوند.

global_asm!

```rust
use std::arch::global_asm;

// اسمبلی در سطح فایل، نه داخل تابع (قالب ELF در Linux)
global_asm!(
    ".global my_symbol",
    "my_symbol:",
    "ret",
);

fn main() {
    println!("Assembly is included");
}
```

کاربردهای واقعی شامل دستورهای ویژه‌ی CPU، دسترسی سطح پایین و بخش‌های آغازین برنامه است. معمولاً intrinsic های `std::arch` یا کتابخانه‌های استاندارد امن‌تر و قابل‌حمل‌ترند.

> **اشتباه رایج:** اعلام‌نکردن درست registerهای تغییرکرده یا انتخاب نادرست option ها می‌تواند باعث رفتار تعریف‌نشده شود.

درس ۶۸: no_std و WebAssembly

فرض این درس این است که Cargo، FFI و `unsafe` را می‌شناسید.

تفاوت std، core و alloc

- `std` : کتابخانه‌ی استاندارد کامل؛ شامل فایل، شبکه، thread و امکانات سیستم‌عامل.
- `core` : امکانات پایه مثل `Option` ، `Result` و عملیات عددی؛ بدون نیاز به سیستم‌عامل.
- `alloc` : نوع‌های نیازمند تخصیص حافظه مانند `Vec` و `String` ؛ به یک allocator نیاز دارد.

`#![no_std]` استفاده‌ی خودکار از `std` را غیرفعال می‌کند؛ این به معنی ممنوع‌بودن تخصیص حافظه نیست (با `alloc` ممکن است).

برنامه‌ی no_std با panic_handler

بدون `std` ، باید خودتان مشخص کنید هنگام `panic` چه شود. برای این‌گونه برنامه‌ها معمولاً در `Cargo.toml` رفتار panic را روی `abort` می‌گذارند:

```
[profile.dev]
panic = "abort"

[profile.release]
panic = "abort"
```

```rust
#![no_std]
#![no_main]

use core::panic::PanicInfo;

// نماد ورودی قابل‌شناسایی برای linker
#[unsafe(no_mangle)]
pub extern "C" fn run() -> i32 {
    42
}

// در این نمونه، panic برنامه را متوقف می‌کند
#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {}
}
```

این نمونه برای محیطی است که runtime و linker مناسب را خودش فراهم می‌کند (مثلاً target های میکروکنترلر یا WebAssembly)؛ به‌تنهایی روی سیستم‌عامل معمولی یک فایل اجرایی مستقل تولید نمی‌کند.

انتخاب target و کامپایل

`wasm32-unknown-unknown` برای ماژول‌های WebAssembly بدون وابستگی مستقیم به سیستم‌عامل مناسب است. `wasm32-wasip1` برای محیط‌های اجرایی که از WASI پشتیبانی می‌کنند (مثل اجرای برنامه خارج از مرورگر) به کار می‌رود.

```
# نصب target ها
rustup target add wasm32-unknown-unknown wasm32-wasip1

# ساخت کتابخانه برای WebAssembly
cargo build --target wasm32-unknown-unknown --lib
cargo build --target wasm32-wasip1
```

اتصال Rust به JavaScript با wasm-bindgen

در `Cargo.toml`، نوع crate را `cdylib` بگذارید و وابستگی `wasm-bindgen` را اضافه کنید:

```
[lib]
crate-type = ["cdylib"]

[dependencies]
wasm-bindgen = "0.2"
```

```
use wasm_bindgen::prelude::*;

// این تابع از JavaScript قابل فراخوانی است
#[wasm_bindgen]
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

برای تولید binding های JavaScript معمولاً از `wasm-pack` یا ابزار `wasm-bindgen` استفاده می‌شود؛ مثلاً `wasm-pack build --target web`.

اشتباه رایج: `wasm32-unknown-unknown` به‌خودی‌خود به معنی اجرای مستقیم فایل در مرورگر نیست؛ برای ارتباط با JavaScript معمولاً به binding و روش بارگذاری مناسب نیاز دارید.

بخش هشتم: مرور نهایی و جدول‌های مرجع

درس ۶۹: مرور نهایی نمادها و کلمات کلیدی Rust

این درس یک cheat-sheet فشرده است: جدول همه‌ی کلمه‌های کلیدی، همه‌ی عملگرها و نشانه‌ها، و انواع comment و literal. هر ردیف یک خط است و به درسی که موضوع را توضیح داده ارجاع می‌دهد.

۱. کلمه‌های کلیدی فعال

کلمه	کاربرد
`as`	تبدیل نوع، نام مستعار در `use`، و مسیر qualified مثل `<T as Trait>`
`async`	تعریف تابع، بلوک یا closure ناهمگام
`await`	انتظار برای تکمیل یک `Future`
`break`	خروج از حلقه، با امکان برگرداندن مقدار
`const`	ثابت زمان کامپایل، `const fn`، const generic، بلوک `const { }` و `*const T`
`continue`	رفتن به تکرار بعدی حلقه
`crate`	ریشه‌ی crate جاری در مسیرها و در `pub(crate)`
`dyn`	trait object با dispatch پویا
`else`	شاخه‌ی جایگزین در `if`، `if let` و `let else`
`enum`	تعریف نوعی با چند variant
`extern`	بلوک خارجی، ABI (مثل `extern "C"`) و `extern crate`
`false`	مقدار منطقی نادرست
`fn`	تعریف تابع و نوع function pointer
`for`	حلقه روی iterator، `impl Trait for Type` و `for<'a>`

کلمه	کاربرد
`if`	اجرای شرطی و match guard
`impl`	پیاده‌سازی متد و trait؛ `impl Trait` در نوع
`in`	`for x in ...`، `pub(in path)` و operand اسمبلی
`let`	تعریف متغیر با pattern، `if let` و `while let`
`loop`	حلقه‌ی بی‌نهایت
`match`	تطبیق الگو و انتخاب شاخه
`mod`	تعریف ماژول
`move`	انتقال مالکیت متغیرها به closure یا async block
`mut`	قابل‌تغییر کردن binding، reference و pointer
`pub`	عمومی کردن آیتم
`ref`	گرفتن reference هنگام تطبیق الگو
`return`	خروج از تابع با مقدار
`self`	گیرنده‌ی متد و مسیر ماژول جاری
`Self`	نوع جاری در trait یا `impl`

کلمه	کاربرد
`static`	متغیر با عمر برنامه؛ `'static` یک lifetime است
`struct`	تعریف ساختار داده
`super`	ماژول والد در مسیر
`trait`	تعریف قرارداد رفتاری
`true`	مقدار منطقی درست
`type`	type alias و associated type
`unsafe`	بلوک، تابع، trait و impl ناامن
`use`	واردکردن نام به محدوده و `use<...>` در bound
`where`	قیدهای نوع و lifetime
`while`	حلقه‌ی شرطی

۲. کلمه‌های رزروشده و contextual

کلمه	وضعیت
`abstract`، `become`، `box`، `do`، `final`، `gen`، `macro`، `override`، `priv`، `try`، `typeof`، `unsized`، `virtual`، `yield`	رزروشده برای استفاده‌ی احتمالی آینده؛ فقط با `r#` به‌عنوان نام قابل استفاده‌اند
`macro_rules`	contextual؛ تعریف declarative macro
`union`	contextual؛ تعریف نوع با فیلدهای هم‌پوشان
`raw`	contextual؛ در `&raw const` و `&raw mut` و گزینه‌ی `raw` اسمبلی
`safe`	contextual؛ علامت‌گذاری آیتم امن داخل `unsafe extern`

۳. عملگرها و نشانه‌ها

نماد	کاربردها
`+`	جمع؛ ترکیب trait bound (`A + B`)؛ تکرار یک‌یاچندبار در ماکرو
`-`	تفریق؛ منفی‌سازی
`*`	ضرب؛ dereference؛ raw pointer (`*const T`)؛ glob import؛ تکرار در ماکرو
`/`	تقسیم
`%`	باقیمانده
`^`	XOR بیتی
`!`	NOT منطقی و بیتی؛ فراخوانی ماکرو (`println!`)؛ inner attribute (`#![...]`)؛ never type؛ negative impl
`&`	AND بیتی؛ borrow؛ reference type (`&T`)؛ reference pattern
`|`	OR بیتی؛ پارامترهای closure؛ or-pattern
`&&`	AND منطقی (lazy)؛ دو borrow پشت‌سرهم؛ reference pattern دوتایی
`||`	OR منطقی (lazy)؛ closure بدون پارامتر
`<<` , `>>`	شیفت بیتی چپ و راست (در generic، `>>` ممکن است دو پرانتز بسته‌ی `>` باشد)
`+= -= *= /= %= ^= &= |= <<= >>=`	انتساب ترکیبی
`=`	انتساب؛ `let`؛ attribute و تعریف نوع‌ها (`type A = B`)

نماد	کاربردها
`== !=`	برابری و نابرابری
`< > <= >=`	مقایسه؛ `<` و `>` بخش generic هم هستند (`Vec<T>`)
`@`	اتصال نام به pattern (`n @ 1..=5`)
`.`	دسترسی به فیلد؛ فراخوانی متد؛ اندیس tuple؛ `.await`
`..`	range نیمه‌باز؛ struct update؛ rest pattern؛ range pattern
`...`	تابع variadic در بلوک خارجی (شکل قدیمی range pattern دیگر مجاز نیست)
`..=`	range شامل انتها (در expression و pattern)
`,`	جداکننده‌ی آرگومان‌ها، فیلدها و عناصر
`;`	پایان statement؛ طول array در `[T; N]` و `[x; n]`
`:`	جداکننده‌ی نام و نوع؛ trait bound؛ فیلد struct؛ نوع fragment در ماکرو
`::`	جداکننده‌ی path؛ turbofish (`::<T>`)؛ دسترسی به associated item
`->`	نوع بازگشتی تابع، closure و function pointer
`=>`	جداکننده‌ی pattern و بدنه در `match` و ماکرو
`<-`	بلااستفاده (توکن شناخته‌شده‌ی بدون معنا)

نماد	کاربردها
`#`	آغاز attribute؛ raw string (`r#"..."#`)
`$`	متغیر و تکرار در `macro_rules!`
`?`	عملگر propagation؛ `?Sized`؛ تکرار اختیاری در ماکرو
`~`	بلااستفاده (توکن شناخته‌شده‌ی بدون معنا)
`_`	wildcard در pattern؛ placeholder نوع و const؛ `use Trait as _`؛ `'_` برای lifetime
`'`	آغاز lifetime یا label حلقه؛ delimiter کاراکتر (`'a'`)
`"`	delimiter رشته
`()`	unit؛ پارامترهای تابع؛ گروه‌بندی؛ tuple
`[]`	array و slice؛ اندیس‌گذاری
`{}`	بلوک؛ بدنه‌ی آیتم؛ struct literal؛ جاگذاری در فرمت‌بندی

۴. Comment ها و literal ها

نوع	نمونه و کاربرد
comment تک‌خطی	`// ...`
comment بلوکی	`/* ... */` (می‌تواند تودرتو باشد)
doc comment بیرونی	`/// ...` و `/** ... */` : مستندسازی آیتم بعدی
doc comment درونی	`//! ...` و `/*! ... */` : مستندسازی crate یا ماژول جاری
عدد صحیح	`42` , `1_000` , `0xff` , `0o77` , `0b1010`
عدد با پسوند نوع	`42u8` , `10_i64` , `1usize`
عدد اعشاری	`3.14` , `1e6` , `2.5f32`
bool	`true` , `false`
کاراکتر	`'A'` , `'\n'` , `'ش'` (یک مقدار Unicode)
رشته	`"Hello\n"` (UTF-8 با escape)
رشته‌ی خام	`r"C:\temp"` , `r#"a "quoted" b"#`
بایت	`b'A'` (یک `u8`)
رشته‌ی بایتی	`b"Hello"` (از نوع `&[u8; N]`)
رشته‌ی بایتی خام	`br"\n"` , `br#"..."#`

نوع	نمونه و کاربرد
رشته‌ی C	`c"hello"` (از نوع `&CStr`)
رشته‌ی C خام	`cr"hello"`، `cr#"..."#`
array تکراری	`[0; 8]`
tuple	`(1, "a")`
unit	`()`
range	`1..5`، `1..=5`، `..`، `1..`، `..5`، `..=5`

ادامه‌ی مسیر

با گذراندن این ۶۹ درس، همه‌ی نحو زبان و مفاهیم اصلی آن را دیده‌اید. برای ادامه:

- مرجع رسمی زبان (The Rust Reference) برای جزئیات دقیق نحو و معنای زبان.
- مستندات کتابخانه‌ی استاندارد (`cargo doc --open` یا doc.rust-lang.org/std) برای API ها.
- Rustonomicon برای مباحث عمیق `unsafe` و مدل حافظه.
- ساخت یک پروژه‌ی واقعی کوچک (مثلاً یک ابزار خط فرمان) که بتوانید مفاهیم کتاب را در آن به کار ببرید.